



Informe Final de Trabajo Profesional de Ingeniería en Informática

Oxidized Neural Orchestra

Sistema distribuido de entrenamiento de modelos de aprendizaje profundo en Rust para implementar y comparar estrategias de distribución como Parameter Server y All-Reduce.

Tutor: Ing. Ricardo Andrés Veiga
rveiga@fi.uba.ar

Co-Tutor y Orientador: Dr. Ing. José Ignacio Alvarez Hamelin
ihameli@fi.uba.ar

Alumnos

Alejo Ordoñez (*Padrón 108397*)
alordonez@fi.uba.ar

Lorenzo Minervino (*Padrón 107863*)
lminervino@fi.uba.ar

Marcos Bianchi Fernández (*Padrón 108921*)
mbianchif@fi.uba.ar

Facultad de Ingeniería, Universidad de Buenos Aires

Índice

Aval del director	5
Agradecimientos	6
1 Resumen	7
2 Palabras clave	8
3 Abstract	9
4 Keywords	10
5 Introducción	11
6 Estado del arte	12
7 Problema detectado	14
8 Solución implementada	15
8.1 Visión general y decisión rectora	15
8.2 Capa de comunicación	16
8.2.1 Protocolo de dos planos	16
8.2.2 Reducción del tráfico	17
8.2.3 Transporte	17
8.2.4 Handles tipados	17
8.3 Motor de redes neuronales	18
8.3.1 La decisión que habilita todo lo demás	18
8.3.2 Componentes	19
8.4 Los tres algoritmos	20
8.4.1 Parameter Server	20
8.4.2 Ring All-Reduce	20
8.4.3 Strategy Switch	21
8.5 Plano de control	22
8.5.1 Selección de topología	22
8.5.2 Sesión orientada a eventos	22
8.6 Interfaces	22
8.7 Infraestructura y experimentación	23
8.8 Estado de la implementación	24
9 Metodología aplicada	25
9.1 Gestión y roles	25
9.2 Proceso de desarrollo	25
9.3 Seguimiento, tickets y gestión del alcance	26
9.4 Automatización	26
9.5 Criterios de aceptación	26

9.6	Artefactos e indicadores	27
10	Herramientas externas utilizadas	28
10.1	Lenguajes y entornos	28
10.2	Bibliotecas de Rust	28
10.3	Bibliotecas de Python	30
10.4	Documentación y edición del informe	30
10.5	Herramientas de inteligencia artificial generativa	30
10.6	Consideración sobre licencias	31
11	Experimentación y validación	32
11.1	Pruebas automatizadas	32
11.2	Entorno de experimentación y reproducibilidad	32
11.3	Estudio I: Parameter Server frente a All-Reduce	34
11.3.1	Pregunta e hipótesis	34
11.3.2	Fundamento	34
11.3.3	Criterio de comparación justa	34
11.3.4	Resultados: convergencia	36
11.3.5	Resultados: throughput y escalabilidad	38
11.3.6	Resultados: presión de comunicación	38
11.3.7	Discusión y matriz de decisión	38
11.3.8	Limitaciones del estudio	43
11.3.9	Conclusión del estudio	43
11.4	Estudio II: impacto de las épocas offline	45
11.4.1	Pregunta e hipótesis	45
11.4.2	Metodología	45
11.4.3	Resultados: tiempo de ejecución	46
11.4.4	Resultados: exactitud	46
11.4.5	Resultados: inestabilidad de la pérdida	47
11.4.6	Discusión y limitaciones	50
11.4.7	Conclusión del estudio	50
11.5	Estudio III: Strategy Switch	52
11.5.1	Pregunta e hipótesis	52
11.5.2	Fundamento	52
11.5.3	Metodología	52
11.5.4	Resultados	52
11.5.5	Discusión, limitaciones y conclusión del estudio	52
11.6	Síntesis de la validación	53
12	Cronograma de las actividades realizadas	54
12.1	Fases efectivamente recorridas	54
12.2	Contraste entre lo planificado y lo ejecutado	54
12.3	Carga horaria efectiva	55
13	Riesgos materializados y lecciones aprendidas	57
13.1	Riesgos previstos que se materializaron	57

13.2 Riesgos no previstos	58
13.3 Desvíos de alcance	59
13.4 Lecciones aprendidas	60
14 Impactos económico, social y ambiental del proyecto	62
15 Trabajos futuros	64
16 Conclusiones	66
17 Aportes individuales	68
18 Referencias	70
19 Anexos	72
19.1 Anexo A: Repositorio del proyecto	72
19.2 Anexo B: Documentación técnica	72
19.3 Anexo C: Reproducibilidad de los experimentos	73
19.4 Anexo D: Monografías	73

Aval del director

En Buenos Aires a [día] de [mes] del año [año] se reúnen el Ing. Ricardo Andrés Veiga con los estudiantes de la carrera de Ingeniería en Informática: el Sr. Alejo Ordoñez (DNI 44480134), el Sr. Lorenzo Minervino (DNI 42720539) y el Sr. Marcos Bianchi Fernández (DNI 43874791). El objetivo de la reunión es revisar el Informe Final del Trabajo Profesional de Ingeniería en Informática.

Luego de haber leído el informe del Trabajo Profesional “Oxidized Neural Orchestra: sistema distribuido de entrenamiento de modelos de aprendizaje profundo en Rust para implementar y comparar estrategias de distribución como Parameter Server y All-Reduce”, el Director del mismo, Ing. Ricardo Andrés Veiga, declara que el mismo se corresponde con el trabajo realizado a lo largo del proyecto y avala el documento presentado como Informe Final, tanto en su completitud como en la claridad de redacción.

Firma y aclaración del director:

Ing. Ricardo Andrés Veiga _____

Firmas y aclaraciones de los estudiantes:

Alejo Ordoñez _____

Lorenzo Minervino _____

Marcos Bianchi Fernández _____

Agradecimientos

A nuestro tutor, el Ing. Ricardo A. Veiga, por acompañar el trabajo de punta a punta y por sostener la exigencia sobre la forma del documento y sobre el rigor de lo que se afirma en él.

A nuestro co-tutor y orientador, el Dr. Ing. José Ignacio Alvarez-Hamelin, y al CoNexDat, Grupo de Redes Complejas y Comunicación de Datos de la Facultad de Ingeniería, por el marco en el que se desarrolló la Práctica Profesional Supervisada y por orientar las decisiones de diseño del sistema distribuido.

A nuestras familias y amigos, que bancaron los dos cuatrimestres que llevó este trabajo.

1 Resumen

El entrenamiento de modelos de aprendizaje profundo requiere de grandes recursos computacionales y largos tiempos de ejecución; los algoritmos que se utilizan hacen uso intensivo de CPU y requieren de capacidades de memoria muy grandes para poder almacenar tanto los datos de entrenamiento, como el modelo en sí. Junto con la revolución de la aplicación de la inteligencia artificial, se ha vuelto un tema candente el minimizar este tiempo de entrenamiento. Para lograrlo, la estrategia más escalable es su ejecución distribuida.

Distribuir el entrenamiento, sin embargo, no es directo: la estrategia de distribución que se elija condiciona tanto el tiempo total de ejecución como la convergencia de la optimización. En este trabajo se implementó *Oxidized Neural Orchestra* (O.N.O.), un sistema distribuido de entrenamiento escrito desde cero en Rust, que incluye una biblioteca de redes neuronales propia construida directamente sobre la biblioteca de arreglos numéricos *ndarray*, sin recurrir a ningún framework de aprendizaje profundo existente. Sobre esa base se implementaron los tres algoritmos que sirven de referencia en el área, *Parameter Server*, *All-Reduce* en anillo y *Strategy-Switch*, junto con tres interfaces de uso: una interfaz interactiva de terminal, una biblioteca de Python construida sobre una interfaz de funciones externas, y un binario de ejecución desatendida.

El sistema resultante es parametrizable y permitió comparar las estrategias entre sí bajo criterios de comparación controlados. Los estudios experimentales realizados sobre MNIST y FashionMNIST muestran que, en un entorno homogéneo y sincrónico simulado sobre una única máquina, *All-Reduce* y *Parameter Server* convergen a la misma exactitud, mientras que *All-Reduce* la alcanza en menos tiempo porque el servidor central introduce un punto de serialización; y que postergar la sincronización entre nodos mediante épocas offline degrada la exactitud final de forma monótona sin aportar, en ese entorno, la ganancia de tiempo que la motiva.

2 Palabras clave

Aprendizaje profundo, entrenamiento distribuido, sistemas distribuidos, paralelismo de datos, *Parameter Server*, *All-Reduce*, *Strategy-Switch*, convergencia, escalabilidad, Rust.

3 Abstract

Training deep learning models requires large computational resources and long execution times; the algorithms involved make intensive use of the CPU and demand very large memory capacities in order to store both the training data and the model itself. Together with the revolution in the application of artificial intelligence, minimizing this training time has become a pressing topic. To achieve this, the most scalable strategy is distributed execution.

Distributing the training, however, is not straightforward: the distribution strategy that is chosen conditions both the total execution time and the convergence of the optimization. In this work we implemented *Oxidized Neural Orchestra* (O.N.O.), a distributed training system written from scratch in Rust, including a neural network library of our own built directly on top of the numerical array library *ndarray*, without relying on any existing deep learning framework. On that foundation we implemented the three algorithms that serve as a reference in the area, *Parameter Server*, ring *All-Reduce* and *Strategy-Switch*, together with three user interfaces: an interactive terminal user interface, a Python library built on a foreign function interface, and a headless binary.

The resulting system is parameterizable and made it possible to compare the strategies against each other under controlled fairness criteria. The experimental studies carried out on MNIST and FashionMNIST show that, in a homogeneous and synchronous environment simulated on a single machine, *All-Reduce* and *Parameter Server* converge to the same accuracy, while *All-Reduce* reaches it in less time because the central server introduces a serialization bottleneck; and that delaying synchronization between nodes through offline epochs degrades final accuracy monotonically without providing, in that environment, the time gain that motivates it.

4 Keywords

Deep learning, distributed training, distributed systems, data parallelism, *Parameter Server*, *All-Reduce*, *Strategy-Switch*, convergence, scalability, Rust.

5 Introducción

El entrenamiento de modelos de aprendizaje profundo requiere de grandes recursos computacionales y largos tiempos de ejecución, y la estrategia más escalable para reducirlos es su ejecución distribuida. Sin embargo, distribuir el entrenamiento no es directo: introduce desafíos en el diseño de la estrategia de distribución y puede degradar la convergencia de la optimización si no se realiza con cuidado. Este trabajo se propuso estudiar en profundidad las principales estrategias de entrenamiento distribuido y, a partir de dicho análisis, implementar un sistema distribuido de entrenamiento de modelos de aprendizaje profundo, incluida su biblioteca de redes neuronales, desarrollada desde cero, que sirviera como base para la comparación de esas estrategias y como punto de partida para el desarrollo de posibles mejoras.

El resultado es *Oxidized Neural Orchestra* (en adelante O.N.O.), un sistema distribuido escrito íntegramente en Rust que implementa tres algoritmos de entrenamiento distribuido, *Parameter Server*, *All-Reduce* en anillo y *Strategy-Switch*, sobre una infraestructura de comunicación y un motor de redes neuronales propios. Sobre ese sistema se realizaron los estudios experimentales que constituyen la validación del trabajo.

Conviene señalar desde el comienzo una particularidad de este informe. A diferencia de un trabajo cuyo objetivo es resolver un problema de negocio, aquí el producto es un *sistema de experimentación*: el valor no está solo en que el sistema funcione, sino en la evidencia que permite producir. Por eso la sección de experimentación ocupa un lugar central y está organizada como tres estudios comparativos independientes, cada uno con su propia pregunta de investigación, su metodología y sus conclusiones acotadas.

El presente documento se organiza de la siguiente manera. En el *Estado del arte* se relevan los avances relacionados al entrenamiento distribuido y se presentan los enfoques fundamentales que sirven de base al trabajo. En *Problema detectado* se precisa la dificultad central que se aborda. En *Solución implementada* se describe en detalle el sistema construido, su arquitectura y las decisiones de diseño que la explican. Luego, en *Metodología aplicada* se detalla el proceso de desarrollo efectivamente seguido, y en *Herramientas externas utilizadas* se declaran las tecnologías empleadas y el criterio con que fueron elegidas. En *Experimentación y validación* se presentan los tres estudios comparativos realizados sobre el sistema. Siguen el *Cronograma de las actividades realizadas*, los *Riesgos materializados y lecciones aprendidas*, los *Impactos económico, social y ambiental*, los *Trabajos futuros* y las *Conclusiones*. Finalmente se incluyen los *Aportes individuales*, las *Referencias* y los *Anexos*.

6 Estado del arte

El entrenamiento distribuido surge como una extensión de las técnicas de paralelización en *HPC* (High Performance Computing). Con los primeros enfoques de paralelismo de datos en frameworks como **TensorFlow** (Developers, 2021) y **PyTorch** (Paszke et al., 2019), y sus implementaciones más recientes optimizadas para arquitecturas heterogéneas, la evolución de estos métodos refleja la tensión constante entre escalabilidad y coherencia de los parámetros del modelo. Los relevamientos de Ben-Nun y Hoefer (Ben-Nun & Hoefer, 2019) y de Dehghani y Yazdanparast (Dehghani & Yazdanparast, 2023) sistematizan esa tensión y sirvieron de mapa conceptual para este trabajo.

El equilibrio entre convergencia y eficiencia de comunicación ha motivado una amplia línea de investigación. En la actualidad, existen dos enfoques fundamentales que sirven como base para el desarrollo de nuevas técnicas: **Parameter Server** (M. Li et al., 2014) y **All-Reduce** (Z. Li, Davis, & Jarvis, 2017).

En el enfoque de Parameter Server, el modelo se divide en un conjunto de parámetros centralizados (no necesariamente en un único nodo), a los cuales los nodos trabajadores envían sus gradientes y desde los cuales reciben las actualizaciones, que surgen de la agregación de estas contribuciones. Este esquema admite tanto una coordinación sincrónica, en la que el servidor espera los gradientes de todos los trabajadores antes de actualizar el modelo, como una asincrónica, en la que aplica cada gradiente apenas llega; esta última mejora la escalabilidad y reduce el tiempo de entrenamiento, aunque a costa de la coherencia entre copias del modelo. El caso extremo de esa relajación es *Hogwild!* (Niu, Recht, Ré, & Wright, 2011), que directamente abraza las condiciones de carrera sobre los parámetros compartidos y demuestra que, bajo hipótesis de esparsidad, la convergencia se preserva.

En contraste, en All-Reduce todos los nodos intercambian gradientes de manera directa y avanzan de forma sincrónica: en cada paso todos aplican la misma actualización, lo que preserva la coherencia entre réplicas. El precio es el costo de comunicación, que crece con el número de nodos, aunque topologías como el anillo lo atenúan. La formulación en anillo de Patarasuk y Yuan (Patarasuk & Yuan, 2009) es óptima en ancho de banda, ya que el volumen que comunica cada nodo resulta esencialmente independiente de la cantidad de participantes, y es la que popularizó **Horovod** (Sergeev & Del Balso, 2018) en el ámbito del aprendizaje profundo. Trabajos posteriores como **BytePS** (Jiang et al., 2020) muestran que, en clústeres heterogéneos, la elección entre ambas familias deja de ser evidente.

Uno de los claros casos de combinación de estos algoritmos es **Strategy-Switch** (Provatas, Chalas, Konstantinou, & Koziris, 2025), que inicia el entrenamiento de forma sincrónica sobre All-Reduce y, guiado por una regla empírica, continúa con Parameter Server asincrónico una vez que el modelo en entrenamiento se estabiliza; logrando así combinar la precisión del régimen sincrónico de *All-Reduce* con la reducción de tiempo del régimen asincrónico de *Parameter Server*. Este algoritmo es el más reciente de los tres y fue el que motivó originalmente la elección del tema.

Una línea transversal a los tres enfoques es la **reducción del volumen de comunicación**. Aji y Heafield (Aji & Heafield, 2017) proponen transmitir únicamente los componentes de mayor magnitud del gradiente y acumular localmente el residuo, y Lin et al. (Lin, Han, Mao, Wang, & Dally, 2018) extienden la idea con compensaciones que preservan la convergencia a tasas de compresión muy altas. Una alternativa complementaria es reducir la precisión numérica de los mensajes sin alterar la precisión del cómputo. Ambas técnicas fueron implementadas en este trabajo.

Finalmente, en el terreno del aprendizaje federado, McMahan et al. (McMahan, Moore, Ramage, Hampson, & Arcas, 2017) formalizan la idea de dejar que cada nodo entrene de forma autónoma durante varias épocas antes de sincronizar, y caracterizan el fenómeno de divergencia entre réplicas que esto induce, conocido como *client drift*. Ese mecanismo, que en O.N.O. se expone bajo el nombre de *épocas offline*, es el objeto de uno de los estudios experimentales de este informe.

7 Problema detectado

El principal desafío del desarrollo de algoritmos distribuidos de entrenamiento de modelos de aprendizaje profundo es encontrar una estrategia que disminuya satisfactoriamente el tiempo del entrenamiento, o que provea una solución a la incapacidad de un flujo existente de almacenar el dataset o incluso el modelo en sí. Esto sería mucho más sencillo si las iteraciones no fueran dependientes; podríamos simplemente utilizar una estrategia análoga a un *fork-join* y combinar la solución. Pero en un *gradient-descent*, la dirección de un descenso en la superficie de costo depende del punto en el que se encuentra el proceso en un momento dado, es decir, depende de la configuración de la matriz de pesos del modelo en esa iteración.

En un esquema de paralelización del entrenamiento por datos, existe el riesgo de incurrir en *overfitting* sobre las particiones si los pesos no se sincronizan con la frecuencia suficiente. Por otro lado, si la sincronización se realiza con demasiada frecuencia, el tiempo de comunicación entre los nodos puede volverse dominante. Este costo no es en absoluto despreciable: si la comunicación representa una fracción significativa del tiempo total de entrenamiento, la distribución del cómputo pierde sentido, ya que la ejecución paralela termina siendo más lenta que el entrenamiento secuencial.

Adicionalmente, no alcanza con dejar que cada nodo optimice por separado hasta una solución especializada sobre su partición y luego combinar esos modelos mediante un promedio de sus pesos, ya que el resultado de cada iteración de los algoritmos de optimización que se utilizan para el entrenamiento depende del anterior y esas soluciones especializadas no son directamente combinables. Por eso las estrategias distribuidas sincronizan durante el entrenamiento, agregando los gradientes de los nodos en cada paso en lugar de promediar modelos ya entrenados por separado.

Este equilibrio entre convergencia y eficiencia de comunicación es, en definitiva, el problema que este trabajo aborda. Ahora bien, sobre ese problema general se recorta una dificultad más concreta, que es la que motivó la construcción del sistema. La literatura describe las estrategias de distribución y reporta resultados sobre ellas, pero **comparar esas estrategias entre sí de forma controlada es difícil**. Las implementaciones de referencia disponibles están optimizadas para escenarios de producción, arrastran décadas de trabajo de ingeniería que no es uniforme entre estrategias y ocultan detrás de capas de abstracción exactamente aquellas decisiones que se querría poder variar: el régimen de sincronización, la ubicación del optimizador, el esquema de particionado de los parámetros, la codificación de los mensajes. Al comparar dos estrategias sobre dos implementaciones distintas, la diferencia medida mezcla el efecto del algoritmo con el de su calidad de implementación.

Se detectó entonces la ausencia de una **base común, parametrizable y de código abierto**, sobre la cual las distintas estrategias de distribución compartan la misma capa de comunicación, el mismo motor de redes neuronales y el mismo plano de control, de modo que la única variable entre dos ejecuciones sea la estrategia bajo estudio. Sin esa base, cualquier comparación entre algoritmos, y en particular la evaluación de mejoras propuestas sobre ellos, queda expuesta a un sesgo que no se puede cuantificar. Construir esa base, e instrumentarla para que produzca mediciones, es el problema concreto que este trabajo resuelve.

8 Solución implementada

Se construyó **Oxidized Neural Orchestra** (O.N.O.), un sistema distribuido de entrenamiento de modelos de aprendizaje profundo escrito íntegramente en Rust. El sistema comprende una biblioteca de redes neuronales desarrollada desde cero, una capa de comunicación con protocolo propio, un plano de control que coordina la ejecución, los tres algoritmos de distribución y tres interfaces de uso.

Esta sección describe la arquitectura resultante. El énfasis está puesto en las **decisiones de diseño y en las razones que las motivaron**, por encima de los detalles de implementación: en un trabajo cuyo objetivo era construir una base de comparación, son esas decisiones las que determinan si la base sirve o no para lo que fue construida.

8.1 Visión general y decisión rectora

El sistema se organiza como un *workspace* de Cargo con ocho crates y unas 27 000 líneas de código, más una suite de experimentación en Python. La dependencia entre ellos es estrictamente jerárquica: `comms` no depende de ningún otro crate del sistema, `machine_learning` depende únicamente de `comms` y no contiene nada relativo a redes, y sobre ambos se construyen los runtimes de los nodos y el plano de control.

Crate	Rol
<code>comms</code>	Capa de red: protocolo, transporte, handles tipados, compresión, reparto del dataset
<code>machine_learning</code>	Motor de redes neuronales, sin networking
<code>parameter_server</code>	Runtime del servidor de parámetros: almacenamiento y sincronización
<code>worker</code>	Runtime del nodo trabajador en sus dos variantes, más los middlewares de topología
<code>orchestrator</code>	Esquema de configuración, validación, cálculo de topología y sesión de entrenamiento
<code>node</code>	El único binario desplegable, agnóstico del rol
<code>orchestui</code>	Interfaz interactiva de terminal
<code>orchestra-py</code>	Biblioteca de Python sobre la interfaz de funciones externas

La **decisión rectora de toda la arquitectura es el agnosticismo de rol**. Todas las máquinas del clúster ejecutan el mismo binario, `node`; el rol que cada una cumplirá, trabajador o servidor de parámetros, se determina en tiempo de ejecución a partir de la especificación que el orquestador le entrega al conectarse. La configuración de un entrenamiento lleva una **única lista plana de direcciones** más la cantidad de servidores deseada, y nunca declara qué dirección será servidor: eso lo decide el orquestador a partir de las latencias que mide.

Esta decisión se tomó frente a la alternativa evidente, que era mantener binarios separados de trabajador y de servidor. Se eligió la unificación porque el sistema debía poder **cambiar la topología en tiempo de ejecución**: Strategy-Switch requiere que un nodo que arrancó como trabajador se

convierta en servidor a mitad del entrenamiento, y eso es imposible si el rol está fijado en el binario que se desplegó. El costo de la decisión fue técnico y concreto: obligó a usar tareas de ejecución local en el runtime asincrónico, porque uno de los runtimes contiene un objeto de entrenamiento que no puede moverse entre hilos.

Una segunda decisión estructural, tomada muy temprano y sostenida hasta el final, es la **separación estricta entre infraestructura y dominio**. El trabajador es “estrictamente infraestructura”: no sabe nada de redes neuronales, es genérico sobre una estrategia de entrenamiento, y todo lo relativo al aprendizaje vive detrás de contratos compartidos. La contracara de esa regla, del lado de la red, es que **los trucos de codificación y de precisión viven exclusivamente en comms**. Esta última regla fue la más disputada del proyecto y llegó a rechazarse una implementación completa de codificación en media precisión por violarla, filtrando tipos de la biblioteca de media precisión hacia otros módulos. El diseño finalmente aceptado establece que los trabajadores y los servidores nunca tocan media precisión y siempre operan en precisión simple; la conversión ocurre únicamente en el serializador y el deserializador.

8.2 Capa de comunicación

8.2.1 Protocolo de dos planos

La decisión de diseño central de la capa de comunicación es un **protocolo híbrido**, que separa el plano de control del plano de datos y los serializa de manera completamente distinta.

El **plano de control** transporta veinte variantes de comando: establecimiento de conexión, creación de nodo con su especificación, medición de latencia, reporte de pérdidas, pedido de parámetros, reparto del dataset, orden de detención, conmutación de estrategia, promoción de un nodo a servidor y desconexión. Son mensajes poco frecuentes y estructuralmente ricos, de modo que se serializan como JSON: la ergonomía y la evolucionabilidad del formato pesan más que los bytes.

El **plano de datos** transporta gradientes, parámetros y fragmentos del conjunto de datos. Aquí el criterio se invierte por completo: son mensajes enormes y frecuentes, y se serializan **reinterpretando la memoria directamente**, sin ninguna biblioteca de serialización de por medio. El resultado es un camino de datos sin copias.

Esa ausencia de copias no es una afirmación de intención sino una propiedad construida deliberadamente, y se apoya en tres detalles que vale la pena señalar porque muestran el nivel al que se trabajó el problema. El primero es que el envío emite una **única llamada al sistema vectorizada** sobre el encabezado y el tensor, de modo que el tensor nunca se copia a un buffer intermedio. El segundo es que el buffer de recepción está respaldado por un vector de enteros de 32 bits en lugar de bytes, **con el único propósito de garantizar el alineamiento a cuatro bytes** que la reinterpretación de memoria requiere para ser válida. El tercero es que el mensaje de parámetros es un préstamo **mutable**, lo que permite al receptor modificar los parámetros en el propio buffer de lectura sin copiarlos.

Un detalle menor pero elegante: el indicador que marca el último mensaje de gradiente de una fase no ocupa ningún byte, ya que se codifica en el bit menos significativo del campo que identifica el tipo de mensaje.

Esa señalización merece una nota, porque es una decisión de diseño generalizable a la que el proyecto

llegó por la vía del error. Originalmente el trabajador señalizaba el fin del entrenamiento **cerrando el socket**, y el servidor leía ese cierre como una falla de escritura. La corrección movió la señal al protocolo: **el fin de una fase distribuida es información de aplicación, no un evento de transporte**.

8.2.2 Reducción del tráfico

Se implementaron dos técnicas complementarias para reducir el volumen de datos que circula por la red.

La primera es la **codificación de gradientes en media precisión**, que reduce a la mitad los bytes transmitidos. Se aplica únicamente a los gradientes; los parámetros y los fragmentos de datos viajan en precisión simple.

La segunda es un **protocolo de gradientes dispersos** que sigue el enfoque de Aji y Heafield (Aji & Heafield, 2017): se transmiten únicamente los componentes de mayor magnitud del gradiente y los descartados se acumulan en un residuo que se considera en el mensaje siguiente. El umbral que separa unos de otros no se calcula de forma exacta, ya que hacerlo exigiría ordenar el gradiente completo en cada paso; en su lugar se **estima muestreando** hasta 16 384 valores y aplicando una selección parcial de costo lineal. El formato serializado codifica tramos contiguos por encima del umbral mediante desplazamientos relativos, lo que evita transmitir un índice por cada valor.

La decisión más interesante de este componente, sin embargo, no es ninguna de las dos técnicas sino **cómo se eligen entre sí**. La compresión es **adaptativa**: se calcula la versión dispersa y solo se utiliza si su tamaño resulta efectivamente menor que el de la versión densa en media precisión; en caso contrario se degrada limpiamente a esta última. Un gradiente denso, que es lo que produce una red convolucional, no se beneficia de la codificación dispersa, y el sistema lo detecta en lugar de suponerlo.

8.2.3 Transporte

El transporte está definido como un *trait* asíncronico con dos operaciones, envío y recepción, pensado para ser apilado siguiendo el patrón decorador. Durante buena parte del proyecto tuvo tres capas: enmarcado de mensajes, vencimiento por tiempo y reintento. **Al cierre del trabajo quedó una sola**, la de enmarcado, y las otras dos fueron eliminadas.

Esta reversión se describe en detalle en *Riesgos materializados y lecciones aprendidas*, porque su razón es una lección más que una decisión técnica aislada. En resumen: los vencimientos enmascaraban errores reales en lugar de mitigarlos, declarando muerto a un nodo que simplemente estaba ocupado, y reintentar sobre una conexión ya muerta no aporta nada sin un protocolo de reconexión que nunca se logró construir. El sistema hereda hoy su fiabilidad de TCP, y el *trait* permanece como costura para retomar el trabajo. La consecuencia es explícita y se declara sin rodeos: **O.N.O. no tolera fallos de nodos**.

8.2.4 Handles tipados

Un rasgo de diseño que atraviesa toda la capa de comunicación es el uso del sistema de tipos para hacer **irrepresentables los estados inválidos**. Cada rol tiene su propio handle con únicamente

las operaciones que le corresponden. La asignación de rol a un nodo recién conectado **consume** el handle sin rol y devuelve uno de trabajador o de servidor, lo que hace imposible por construcción operar sobre un nodo al que todavía no se le asignó un papel. El mismo mecanismo sostiene la promoción en Strategy-Switch: el handle de trabajador se consume y devuelve un handle de servidor reutilizando la misma conexión, de modo que la transición no puede dejar al sistema en un estado intermedio.

La misma filosofía aparece en los valores numéricos. Los hiperparámetros que tienen rangos válidos acotados, como una probabilidad o una tasa de aprendizaje positiva, se representan mediante tipos que validan en el borde de deserialización. Un valor inválido no llega nunca al interior del sistema.

8.3 Motor de redes neuronales

El motor de aprendizaje se implementó por completo en el marco de este trabajo, sobre una biblioteca que provee únicamente operaciones sobre arreglos numéricos n-dimensionales. La propagación hacia adelante y hacia atrás, los optimizadores, las funciones de pérdida y la representación de los tensores son propios.

8.3.1 La decisión que habilita todo lo demás

La decisión de diseño más consecuente del motor es que **las capas no almacenan sus parámetros**. Los parámetros se pasan como un fragmento plano de memoria en cada llamada, y la capa los reinterpreta con la forma que le corresponde.

Vale la pena detenerse en por qué esto importa, porque su alcance excede al motor de aprendizaje. El modelo completo vive en un **único buffer plano**, sobre el cual las capas operan como una vista sin estado definida por desplazamientos y formas. Esa representación es exactamente lo que permite tres cosas que de otro modo serían imposibles: que la entrada y salida de red no requiera copias, ya que el buffer del modelo es directamente el buffer que se transmite; que el modelo pueda **partirse entre varios servidores** entregándole a cada uno un rango del buffer; y que la misma implementación del modelo funcione sin cambios tanto en un régimen donde una sola entidad posee todas las capas como en uno donde están repartidas.

Esa última propiedad se materializa en el componente que unifica los tres algoritmos, que no serializa nada: simplemente rebana uno o varios buffers planos y entrega a cada capa su ventana. Bajo All-Reduce, una sola entidad posee todas las capas y cada nodo tiene una réplica completa. Bajo Parameter Server, un vector de asignación indica a qué servidor pertenece cada capa. **El mismo modelo secuencial corre sin modificaciones en ambos regímenes**, y esa es la verdadera costura de polimorfismo entre algoritmos del sistema.

Como se explica en la sección de riesgos, esta decisión tuvo un costo que solo se hizo visible mucho después: el préstamo de memoria que habilita la ausencia de copias es el que impidió construir la capa de reconexión.

Una segunda decisión, complementaria, es que **las capas poseen sus buffers de salida y devuelven vistas hacia sí mismas**. Después del calentamiento inicial, una pasada de entrenamiento no reserva memoria dinámica.

8.3.2 Componentes

Las capas se modelan como un enumerado cerrado con despacho estático y no como un *trait* con despacho dinámico, decisión coherente con el objetivo de rendimiento. Se implementaron capas densas, convolucionales, de *max-pooling*, de reordenamiento de forma y las activaciones sigmoide, tangente hiperbólica, ReLU y softmax. La capa de reordenamiento, que es de costo nulo, es el adaptador que permite convivir a las capas densas con las convolucionales, y el constructor la **inserta automáticamente** en las fronteras correspondientes, de modo que el lenguaje de configuración no necesita exponer el concepto.

La convolución merece una mención aparte porque concentró buena parte del esfuerzo del trabajo. Su implementación final está basada en la reformulación de la convolución como una **multiplicación de matrices**, lo que permite delegar el trabajo pesado en las rutinas de álgebra lineal optimizadas y aprovechar la localidad de caché. Se llegó a ella después de probar y descartar dos alternativas, la transformada rápida de Fourier y la paralelización con hilos, según se detalla en las lecciones aprendidas. La mejora resultante fue de 2,27 veces en tiempo total, con la exactitud subiendo de 97,26 % a 98,5 %.

Se implementaron dos funciones de pérdida, error cuadrático medio y entropía cruzada, ambas calculando el valor y el gradiente en una sola pasada fusionada. La entropía cruzada acota su argumento para protegerse del logaritmo de cero. Corresponde señalar una **brecha de optimización conocida**: softmax es una capa ordinaria cuyo jacobiano completo se compone con el gradiente de la entropía cruzada, en lugar de estar fusionados. Matemáticamente el resultado es equivalente, pero cuesta una pasada adicional por fila y reintroduce una división acotada que la forma fusionada existe precisamente para evitar.

Los optimizadores implementados son descenso por gradiente, descenso con momento clásico y Adam (Kingma & Ba, 2014; Sutskever, Martens, Dahl, & Hinton, 2013). Las inicializaciones cubren los esquemas de Xavier-Glorot (Glorot & Bengio, 2010), Kaiming-He (He, Zhang, Ren, & Sun, 2015) y LeCun, en sus variantes uniforme y normal. Los inicializadores no son generadores por tensor sino **generadores con presupuesto** que producen valores sobre demanda, lo cual es la consecuencia natural de que el modelo sea un buffer plano; existe además un mecanismo que cose pedidos que cruzan la frontera entre dos generadores, y es el que permite inicializar cada capa con su esquema propio sobre una única tira de memoria.

Una decisión sutil de ubicación merece señalarse: los **optimizadores con estado viven en la frontera de sincronización, no en la del lote local**. El bucle interno por lote usa optimizadores sin estado que replican la tasa de aprendizaje, mientras que los momentos de Adam o de momento clásico solo se aplican al agregar gradientes. La razón es que, de otro modo, en All-Reduce los buffers internos del optimizador contendrían basura al momento de la reducción, y en Parameter Server habría que transmitir el estado del optimizador por mensaje.

El modelo entrenado se exporta en formato **safetensors**, siguiendo la convención de nombres de PyTorch para facilitar la interoperabilidad. Conviene advertir que el layout interno de las capas densas es la traspuesta del que usa PyTorch, de modo que un cargador debe transponer.

8.4 Los tres algoritmos

8.4.1 Parameter Server

El sistema implementa un Parameter Server **multi-servidor con particionado del modelo**. La decisión de diseño relevante es **cómo se reparten las capas** entre los servidores, y se tomó descartando la alternativa obvia.

El particionado equitativo por cantidad de parámetros produce un problema: los parámetros de una misma capa pueden quedar a caballo de dos servidores, lo que hace que el fragmento deje de ser contiguo y fuerce una reserva de memoria. Se optó por **particionar respetando los límites de capa**, repartiendo las capas completas entre los servidores mediante un algoritmo voraz de empaquetado que asigna primero las más grandes. Junto con la asignación viaja al trabajador un vector de orden que le indica, para cada capa, a qué servidor debe pedirle su fragmento.

Esa decisión tiene una consecuencia en el reensamblado final del modelo que no es evidente: como las capas se reparten por tamaño y no por orden, concatenar ingenuamente lo que devuelve cada servidor produciría un modelo barajado. El reensamblado recorre explícitamente los desplazamientos por capa. Un error precisamente de este tipo, en el que los servidores almacenaban las capas ordenadas por tamaño en lugar de por su posición en el modelo, sobrevivió meses en el código porque todas las redes de prueba eran monótonamente decrecientes y ambos órdenes coincidían por casualidad.

Del lado del servidor, el almacenamiento y la sincronización se abstraen detrás de dos *traits* independientes, lo que convierte el régimen de operación en una **elección de configuración en tiempo de ejecución** en lugar de una decisión de compilación. El almacenamiento con bloqueo está fragmentado internamente y usa **doble buffer**: los trabajadores siguen acumulando sobre el buffer activo mientras el congelado se consume, y la actualización usa una operación atómica de comparación e intercambio que la vuelve de ejecución única, de modo que si otro hilo ya está actualizando la llamada no hace nada. El sincronizador con barrera es el que da la semántica sincrónica utilizada en los experimentos.

La barrera merece una nota, porque resuelve el problema de coordinación distribuida más costoso del proyecto. Es una **barrera de tamaño dinámico**: un trabajador que termina su entrenamiento y se retira decrementa permanentemente el contador, y si con eso la barrera se completa, la avanza él mismo antes de irse. Esa lógica está atada a la destrucción del objeto, de modo que ocurre necesariamente. Es la solución al problema canónico de **membresía estática frente a terminación dinámica**, que con un contador fijo dejaba a los supervivientes esperando indefinidamente a alguien que ya se había ido.

La segunda variante de almacenamiento, sin bloqueos y siguiendo el enfoque de *Hogwild!* (Niu et al., 2011), está implementada pero **deshabilitada**, por las razones de corrección y de validez de supuestos que se detallan en las lecciones aprendidas. No se la considera una variante operativa del sistema.

8.4.2 Ring All-Reduce

La implementación sigue la formulación en anillo de Patarasuk y Yuan (Patarasuk & Yuan, 2009), en dos fases de $n - 1$ pasos cada una. En la primera, cada nodo termina con un fragmento del gradiente reducido sobre todos los participantes; en la segunda, ese fragmento ya reducido se propaga por el

anillo. El costo total es de $2(n - 1)$ pasos, cada uno moviendo aproximadamente $1/n$ del modelo, lo que da el costo óptimo en ancho de banda.

Tres decisiones concretas hacen que funcione. La primera es que el **fragmentado es balanceado**: los fragmentos difieren a lo sumo en un elemento, en lugar de dejar una cola corta, de modo que ningún paso del anillo es sistemáticamente más lento que los demás. La segunda es que **el envío y la recepción de cada paso son concurrentes**. Esto último no fue una decisión de diseño previa sino la corrección de un bloqueo: hacerlos secuencialmente provocaba que todos los nodos intentaran enviar antes de recibir, llenando los buffers de los sockets sin que nadie los drenara. El sistema se colgaba silenciosamente, sin error y sin consumo de CPU, con modelos de casi dos millones de parámetros. La misma lección se aplicó a la construcción del anillo, donde aceptar la conexión entrante y establecer la saliente también se hacen de forma concurrente.

La tercera es que el gradiente reducido **se promedia dividiendo por la cantidad de trabajadores**. Puede parecer un detalle, pero su ausencia fue uno de los tres errores silenciosos que la suite de benchmarks destapó: el anillo sumaba en lugar de promediar, de modo que la tasa de aprendizaje efectiva escalaba con la cantidad de nodos y el entrenamiento divergía al agregar trabajadores.

8.4.3 Strategy Switch

La implementación de Strategy-Switch (Provatas et al., 2025) separa la **decisión**, que vive en el orquestador, de la **ejecución**, que ocurre en los nodos. El criterio de conmutación mantiene una ventana deslizante de las últimas pérdidas y calcula su cambio relativo medio; cuando ese valor cae por debajo de un umbral, es decir, cuando la curva de pérdida **se aplana**, se dispara la transición. La ventana y el umbral son los valores indicados en el trabajo original.

Una decisión de diseño que conviene destacar es que **el plan se precomputa antes de entrenar**. La topología completa del Parameter Server, la ubicación de los servidores y el particionado de las capas se calculan en tiempo de configuración, y para cada nodo se construye por anticipado la acción que le corresponderá. Lo único dinámico es el **momento**. La razón es que calcular la topología requiere las mediciones de latencia, que ya se hicieron al conectar, y hacerlo en caliente introduciría una pausa en el entrenamiento.

La promoción de un trabajador a servidor es real y no una reasignación de etiquetas: el nodo rebana sus **parámetros ya entrenados** según los rangos recibidos y arranca como servidor sembrado con esos pesos, no con valores aleatorios. La ligadura entre cada promoción y su fragmento se hace **por dirección del servidor** y no por orden de aparición en el recorrido de la topología; hacerlo por orden fue un error real que provocaba que el fragmento cayera en el servidor equivocado.

Corresponde declarar tres limitaciones. La transición es **unidireccional**, de All-Reduce a Parameter Server, y **de un solo disparo** por sesión. Y, lo más importante para la lectura de los resultados experimentales, **en la campaña de benchmarks ejecutada el criterio de conmutación nunca llegó a dispararse**: el umbral de estabilidad, tomado con fidelidad del trabajo original, no se alcanza dentro del presupuesto de épocas evaluado. El sistema registra explícitamente esa condición en cada corrida en lugar de ocultarla. Se optó por mantener la fidelidad al paper antes que calibrar el umbral para el régimen propio, y por declarar la consecuencia.

8.5 Plano de control

El orquestador es el plano de control del sistema. Recibe la configuración del entrenamiento, la valida, se conecta a los nodos, mide la topología de la red, deriva de allí toda la asignación de roles y especificaciones, y conduce la sesión.

8.5.1 Selección de topología

Una de las líneas no previstas en el plan original y que resultó de las más interesantes es la **selección de topología a partir de mediciones**. Antes de comenzar, el orquestador realiza rondas de medición de latencia entre todos los pares de nodos y construye con ellas un grafo.

La decisión de diseño relevante es que el peso de cada arista es el **RTT máximo observado y no el promedio**, y que ambos problemas de optimización se plantean sobre el peor caso. El orden del anillo de All-Reduce se resuelve como un problema del viajante, y la ubicación de los servidores de parámetros como un problema minimax que elige los k nodos que minimizan la latencia peor caso hacia los trabajadores. El criterio quedó resumido en la discusión que lo originó: *“es mejor tener 2 conexiones más o menos que una muy buena y una muy mala”*, que es exactamente la diferencia entre optimizar la suma y optimizar el máximo.

Ambos problemas se resuelven con **algoritmos exactos** y no con heurísticas: programación dinámica sobre subconjuntos para el ciclo, y enumeración con poda para la ubicación de servidores. La decisión es defendible dado el régimen al que apunta el sistema, de decenas de nodos, aunque debe señalarse que el algoritmo del ciclo degenera para tamaños grandes.

8.5.2 Sesión orientada a eventos

La ejecución de un entrenamiento se modela como una **sesión orientada a eventos**. El orquestador expone un canal de eventos y mantiene una tarea por trabajador que traduce los mensajes del protocolo en eventos de dominio: pérdidas publicadas, trabajador terminado, entrenamiento completo, nodo promoviéndose. Este diseño reemplazó a un panel de control simulado y es lo que permite que las tres interfaces del sistema consuman exactamente la misma información sin duplicar lógica.

El punto de agregación es el manejo de pérdidas: se registra la última pérdida de cada trabajador, se exige que **todos los trabajadores vivos** hayan reportado, y recién entonces se alimentan los dos criterios que observan la curva, el de detención temprana y el de conmutación de estrategia. Que la condición sea sobre los trabajadores vivos y no sobre una cantidad fija es otra consecuencia del problema de membresía dinámica.

La detención temprana (Prechelt, 1998) se implementó como un criterio de tolerancia: cuando tres pérdidas agregadas consecutivas se mueven por debajo de un umbral configurable, se considera que el entrenamiento convergió y se difunde la orden de detención. Es un criterio de tolerancia y no de paciencia, y la ventana está fijada en el código.

8.6 Interfaces

El sistema expone tres formas de uso, todas sobre el mismo plano de control y **compartiendo el mismo esquema de configuración canónico**, sin duplicarlo. Esa última propiedad no fue

gratuita: el esquema llegó a estar replicado en tres documentaciones distintas que divergieron entre sí, y se resolvió creando un documento de referencia único y eliminando las copias.

La **interfaz interactiva de terminal** permite configurar, lanzar y monitorear un entrenamiento en vivo. Presenta un asistente de configuración con sugerencias y validación, y un panel que muestra la evolución de la pérdida, el progreso por trabajador y una vista de topología. Esa vista de topología tiene un detalle que vale la pena mencionar porque es el único lugar donde el sistema se observa a sí mismo: las partículas que representan el flujo de datos entre nodos **se mueven al ritmo real del entrenamiento**, siguiendo una media móvil del intervalo entre reportes, y permiten ver en vivo cómo un trabajador se convierte en servidor durante una conmutación de estrategia. El entrenamiento corre en un hilo de fondo para que la interfaz nunca se bloquee.

La **biblioteca de Python** se construyó sobre PyO3. La decisión de diseño detrás de ella se tomó descartando una alternativa que era más limpia en teoría: anotar los tipos existentes del orquestador con los atributos de PyO3 detrás de una bandera de compilación, manteniendo una única fuente de verdad. Se descartó por **restricciones duras del compilador**, ya que los atributos de PyO3 no admiten tipos genéricos, las variantes complejas de enumerados requieren formas incompatibles y algunos tipos de error habrían necesitado capacidades adicionales. La solución fue construir un crate envoltorio separado, lo que mantiene el orquestador como Rust puro. El entrenamiento libera el bloqueo global del intérprete y se ejecuta en un hilo dedicado, de modo que el proceso de Python actúa efectivamente como orquestador.

El **binario de ejecución desatendida** completa el conjunto, y es el que utiliza la suite de benchmarks.

8.7 Infraestructura y experimentación

El despliegue se realiza con una **única imagen de Docker**, coherente con el agnosticismo de rol: no hay imagen de trabajador ni imagen de servidor. La construcción es multietapa con caché de dependencias, y un conjunto de scripts genera la configuración del clúster, ajusta la resolución de nombres y levanta el entorno a partir de un único parámetro con la cantidad de nodos.

Corresponde declarar con precisión lo que esta infraestructura **no** hace, porque condiciona todos los resultados de rendimiento del informe: no fija afinidad de núcleos, no limita CPU por contenedor y no emula latencia ni ancho de banda de red. Todos los nodos comparten un host de ocho núcleos sobre loopback. Los tiempos medidos son, por lo tanto, una cota inferior del costo de comunicación.

La suite de experimentación está escrita en Python y organizada de forma declarativa en cuatro campañas que miden convergencia, velocidad de ejecución, velocidad de convergencia y escalabilidad. Persiste los resultados de forma incremental, agrega repeticiones en media y desvío, y **regenera su propia documentación** en cada corrida. Su aspecto más valioso, y el que se considera un aporte metodológico del trabajo, es que documenta explícitamente sus propios criterios de equidad y sus fuentes de ruido: justifica el presupuesto fijo de nodos, aclara qué significa una época en cada campaña, y explica por qué ciertas variantes se comparan por exactitud y no por velocidad, dado que una medición de tiempo aislada tiene un ruido cercano al 25 %.

8.8 Estado de la implementación

El sistema compila sin advertencias y su conjunto de pruebas pasa con 87 pruebas en verde. El motor de aprendizaje propio converge a 0,983 de exactitud sobre MNIST con la red de referencia y a 0,980 con LeNet-5, frente a 0,989 y 0,990 de una implementación de PyTorch con la misma receta, lo que sitúa la implementación propia a alrededor de medio punto porcentual de un framework maduro.

En honor a la precisión, corresponde declarar las limitaciones verificadas al cierre. La desconexión de un servidor provoca la caída del trabajador, ya que ese camino no está implementado. La variante de almacenamiento sin bloqueos tiene un problema de corrección y está deshabilitada. La propagación hacia atrás de la ReLU con pendiente negativa es incorrecta, aunque el caso estándar resulta correcto y ningún modelo evaluado la utiliza. Las dos pruebas de integración existentes carecen de aserciones y funcionan como pruebas de humo. **Y el anillo de All-Reduce, que es el algoritmo más intrincado del sistema, no tiene cobertura de pruebas:** su corrección está verificada por sus resultados de convergencia y no por pruebas unitarias, lo cual es una deuda real y no una decisión.

9 Metodología aplicada

Esta sección actualiza lo planteado en la propuesta, contrastando lo previsto con lo efectivamente ejecutado.

9.1 Gestión y roles

Se estableció un compromiso de 500 horas por estudiante a lo largo de dos cuatrimestres, lo que representa un promedio de unas 16 horas semanales por persona sobre 32 semanas. El Ing. Ricardo A. Veiga cumplió la función de tutor y el Dr. Ing. J. Ignacio Alvarez-Hamelin la de co-tutor y orientador en el campo de desempeño, en el marco del CoNexDat, Grupo de Redes Complejas y Comunicación de Datos de la Facultad de Ingeniería.

Los tres estudiantes participaron de todas las etapas de análisis, implementación y pruebas. Sin perjuicio de ello, cada integrante concentró su foco principal en un conjunto de componentes, según se detalla en *Aportes individuales*. Esa especialización, que no estaba impuesta por la propuesta sino que emergió del desarrollo, tuvo un efecto secundario que conviene señalar: la revisión cruzada de código se volvió el principal mecanismo de difusión de conocimiento entre componentes, y en varios casos las discusiones de revisión terminaron modificando decisiones de diseño y no solo detalles de implementación.

Se mantuvieron reuniones periódicas en formato virtual con los tutores, con la cadencia comprometida en el Acta de Acuerdo de una periodicidad igual o menor a las dos semanas, para informar el avance, definir prioridades y planificar la etapa siguiente.

9.2 Proceso de desarrollo

El método de desarrollo fue **incremental**, organizado en iteraciones con entregables intermedios. La unidad de entrega fue la *pull request*: cada funcionalidad se desarrolló en una rama propia que se integraba a la rama principal previa revisión. A lo largo del proyecto se produjeron **989 commits**, **108 pull requests** y **80 issues**, entre septiembre de 2025 y julio de 2026.

El código y todos los artefactos versionables se gestionaron con **Git** sobre un repositorio alojado en **GitHub**. Los mensajes de los commits siguen la convención de *Conventional Commits*, lo que mantiene un historial legible y trazable. Los entregables intermedios que marcaron el avance fueron, en orden: la capa de comunicación funcionando de punta a punta, el primer entrenamiento distribuido completo con Parameter Server, el motor de redes neuronales convergiendo sobre MNIST, la interfaz de terminal mostrando un entrenamiento en vivo, el segundo algoritmo (All-Reduce en anillo), y finalmente la suite de benchmarks con sus resultados.

Un patrón de proceso que el equipo adoptó de forma sistemática, y que corresponde declarar porque condiciona la lectura del historial, es el de **cerrar una rama de trabajo en curso y abrir una nueva y limpia** en lugar de iterar sobre la existente. Ocurrió en los componentes centrales del sistema: el servidor de parámetros y el worker llegaron a su diseño definitivo después de tres intentos cerrados cada uno, y lo mismo sucedió con la capa de comunicación, la distribución del conjunto de datos, la codificación en media precisión y la abstracción de All-Reduce. Como disciplina produce revisiones acotadas y un historial de la rama principal limpio; su costo, discutido en *Riesgos*

materializados y lecciones aprendidas, es que la traza del razonamiento queda repartida entre pull requests cerradas.

9.3 Seguimiento, tickets y gestión del alcance

El seguimiento del proceso, la gestión de tickets y el registro de errores se realizaron con las *issues* del repositorio en GitHub, categorizadas mediante etiquetas por tipo (*bug*, *enhancement*, *documentation*) y por componente del sistema (*parameter-server*, *comms*, *worker*, *ffi*, *tui*, entre otras). Las issues se usaron además, y esto excedió lo previsto en la propuesta, como **espacio de debate de diseño**: varias de las decisiones de arquitectura documentadas en este informe se resolvieron enteramente en el hilo de comentarios de una issue, con enumeración de alternativas y su descarte razonado. Ese registro fue el insumo principal para reconstruir el proceso.

Los errores se clasificaron por criticidad, distinguiendo los que impedían el avance del trabajo de los que admitían corrección diferida. La gestión del alcance se materializó en dos momentos explícitos: el diferimiento de la tolerancia a fallos hasta tener un producto mínimo funcionando, tomado al inicio bajo el criterio de que *“estas features van al final, una vez que tengamos el MVP andando”*, y una sesión de definición de alcance previa al cierre en la que se descartaron de forma deliberada cinco líneas de trabajo que no iban a llegar. Ambos se detallan en la sección de riesgos.

9.4 Automatización

La construcción y las pruebas del sistema se ejecutan con las herramientas del ecosistema de Rust (*cargo build*, *cargo test*, *cargo clippy* para análisis estático), y los entornos de simulación se levantan de forma automatizada mediante **Docker** y un **Makefile** autodocumentado, lo que hace reproducible el despliegue de las distintas configuraciones de nodos. Para el código Python se incorporó **ruff** como analizador estático y formateador.

El grado de automatización alcanzado debe declararse con honestidad. La construcción, las pruebas, el análisis estático, el levantamiento de entornos multinodo y la ejecución de campañas completas de benchmarks están **totalmente automatizados y son de ejecución desatendida**: la campaña final de 38 configuraciones se ejecutó de un solo comando durante 5 horas y 39 minutos sin intervención. En cambio, la **integración continua no llegó a funcionar**. La propuesta ya la señalaba como una cuestión no definida cuya adopción dependería de la evolución del proyecto; lo que ocurrió es peor que no haberla adoptado: se escribió un flujo de trabajo de integración continua que **nunca se ejecutó porque residía en un directorio con el nombre mal escrito**, y el error recién se detectó en la limpieza final del repositorio, a dos semanas del cierre. La verificación previa a la integración quedó, en los hechos, a cargo de la ejecución local de las pruebas por parte de quien abría la pull request y de quien la revisaba.

9.5 Criterios de aceptación

Cada pull request se sometió a revisión de código por parte de otro integrante antes de integrarse. Una entrega se consideró aceptada cuando se cumplían de forma conjunta tres condiciones: la revisión de código fue aprobada, la totalidad de las pruebas automatizadas asociadas se ejecutaba con éxito, y la funcionalidad requerida quedaba demostrada mediante una prueba de aceptación. Para las funcionalidades que involucran coordinación entre nodos, esa prueba de aceptación consistió en una

ejecución completa de entrenamiento distribuido sobre el entorno simulado en Docker, verificando que la convergencia obtenida fuera consistente con la del entrenamiento secuencial equivalente.

Este criterio funcionó, pero la experiencia del proyecto muestra que era **insuficiente para detectar errores de corrección distribuida silenciosos**, aquellos que no provocan fallos sino que degradan la calidad del modelo. Se retoma en las lecciones aprendidas.

9.6 Artefactos e indicadores

Como artefactos de gestión se llevaron minutas de las reuniones periódicas, el registro del alcance y el registro de riesgos, revisado en las reuniones de seguimiento. A ellos se sumaron dos artefactos técnicos no previstos que resultaron centrales: el **documento canónico del esquema de configuración**, creado para eliminar la divergencia entre tres copias distintas de la misma documentación, y el **borrador de arquitectura**, que sostuvo las discusiones de diseño en las etapas iniciales.

Sobre la calidad del proceso se definieron tres indicadores: el tiempo entre apertura y cierre de una issue, la proporción de pull requests que requieren correcciones tras la revisión, y el desvío entre horas planificadas y efectivamente dedicadas. Este último funcionó también como indicador de costos, dado que el trabajo se desarrolló sobre hardware propio y con herramientas de código abierto, de modo que el costo del proyecto se reduce a las horas-persona invertidas.

Sobre la calidad del producto se consideraron la proporción de pruebas exitosas sobre el total y la cantidad de advertencias reportadas por el análisis estático, esta última mantenida en cero mediante **clippy** de forma sostenida. La **cobertura de pruebas**, en cambio, se relevó de manera irregular y no se sostuvo como indicador continuo. Ese incumplimiento tuvo consecuencias concretas y verificables: la variante de actualización sin bloqueos de los parámetros nunca tuvo cobertura, y el error que la invalidaba permaneció oculto hasta una auditoría tardía; y un error en la propagación hacia atrás del *max-pooling* quedó enmascarado porque las pruebas existentes usaban un único canal y un único elemento de lote.

10 Herramientas externas utilizadas

En cumplimiento de las pautas de ética profesional, se declaran a continuación todas las herramientas empleadas en el desarrollo del trabajo, junto con las consideraciones que motivaron su elección.

La propuesta fijó tres tecnologías desde el inicio, Rust, Python y Docker, y dejó explícitamente abiertas las decisiones restantes, estableciendo tres criterios para tomarlas: que no introdujeran un costo de comunicación o de serialización capaz de distorsionar la comparación entre estrategias, que se integraran con el sistema de construcción y de pruebas del ecosistema de Rust sin requerir dependencias externas a él, y que su licencia fuera de código abierto de modo que el sistema pudiera ser retomado por terceros. Los tres criterios se sostuvieron.

10.1 Lenguajes y entornos

Rust. Se eligió como lenguaje del sistema principal por tres motivos. El primero es el control de bajo nivel que ofrece sin renunciar a garantías de seguridad, lo que resultó determinante para un sistema cuyo camino crítico exige manipular memoria sin copias. El segundo es la *conurrencia sin miedo*: el compilador verifica estáticamente una clase completa de errores de concurrencia. El tercero es la relevancia creciente del lenguaje en el ámbito de los sistemas.

Corresponde evaluar esa elección a posteriori, ya que la sección de lecciones aprendidas la retoma en detalle. Las garantías del lenguaje cumplieron exactamente lo que prometen: ninguna condición de carrera sobre memoria compartida llegó a producción. No cubrieron, ni podían cubrir, los bloqueos de coordinación distribuida ni los errores de derivación de gradientes, que fueron las dos fuentes reales de dificultad. El único punto en el que el sistema salió de las garantías del lenguaje, la implementación sin bloqueos del almacenamiento de parámetros, es también el único que resultó incorrecto.

El ecosistema se utilizó completo: `cargo` para construcción y gestión de dependencias, `cargo test` como arnés de pruebas, `cargo clippy` para análisis estático y `cargo fmt` para formato. No se incorporó ninguna herramienta externa a él, según el criterio fijado en la propuesta.

Python. Se utilizó para tres propósitos: la interfaz de funciones externas del sistema, por ser el lenguaje dominante en aprendizaje profundo; la suite de experimentación y el análisis de resultados; y los scripts de generación y despliegue del entorno contenedizado. El código Python se verifica con `ruff` como analizador estático y formateador.

Docker. Se utilizó para simular la ejecución del sistema sobre múltiples nodos y para automatizar el despliegue. Es la herramienta que hizo posible la experimentación, y también la fuente de la limitación transversal del trabajo, ya que un clúster simulado sobre una única máquina no reproduce la latencia ni el ancho de banda de una red real. Se acompaña de `cargo-chef` para cachear las capas de dependencias en la construcción de la imagen, lo que reduce de forma sustancial el tiempo de reconstrucción.

Git y GitHub. Control de versiones y gestión del proceso: ramas por funcionalidad, *pull requests* con revisión obligatoria, e *issues* para el seguimiento de tickets, errores y debates de diseño.

10.2 Bibliotecas de Rust

La elección de cada una respondió a los criterios enunciados. Todas son de código abierto.

Biblioteca	Uso
<code>tokio</code>	Runtime asincrónico, TCP, tareas y canales.
<code>ndarray</code>	Base de todo el sistema distribuido
<code>ndarray-rand</code>	Operaciones sobre arreglos numéricos n-dimensionales; provee la multiplicación de matrices del motor
<code>rayon</code>	Inicialización aleatoria de tensores
<code>serde</code> y <code>serde_json</code>	Paralelismo de datos en el almacenamiento del servidor de parámetros
<code>bytemuck</code>	Serialización del plano de control y de los archivos de configuración
<code>half</code>	Reinterpretación de memoria sin copias entre valores numéricos y bytes
<code>uuid</code>	Tipo de media precisión para la compresión de gradientes
<code>futures</code>	Identidad estable de nodos
<code>rand</code> y <code>rand_distr</code>	Composición concurrente de operaciones de red
<code>parking_lot</code>	Muestreo del umbral disperso, barajado y distribuciones de inicialización
<code>trait-variant</code> y <code>async-trait</code>	Primitivas de sincronización de la barrera dinámica
<code>safetensors</code>	Traits asincrónicos
<code>log</code> y <code>tracing-subscriber</code>	Exportación del modelo entrenado
<code>ratatui</code> y <code>crossterm</code>	Registro estructurado con filtrado por variable de entorno
<code>pyo3</code>	Interfaz interactiva de terminal
<code>approx</code>	Interfaz de funciones externas con Python
<code>anyhow</code>	Comparaciones aproximadas en las pruebas
	Manejo de errores en la interfaz de terminal

Tres de estas elecciones merecen justificarse porque tenían alternativas reales.

`ndarray` en lugar de un framework de aprendizaje profundo. Es la decisión que define el alcance del trabajo. Se eligió una biblioteca que provee *únicamente* operaciones sobre arreglos numéricos, sin diferenciación automática, sin capas y sin optimizadores, porque el objetivo del trabajo incluía implementar el motor de aprendizaje desde cero. Usar un framework existente habría hecho el trabajo considerablemente más simple y considerablemente menos valioso.

`bytemuck` para la serialización del plano de datos. Aquí operó de forma directa el primer criterio de la propuesta. Una biblioteca de serialización de propósito general habría introducido una copia y un costo de codificación en el camino crítico, exactamente el tipo de sobrecarga que podría distorsionar la comparación entre estrategias que comunican de maneras distintas. La reinterpretación de memoria elimina esa variable.

`tracing-subscriber` en reemplazo de `env_logger`. Se migró tardíamente, manteniendo la fachada genérica de registro en las bibliotecas. La razón fue concreta: la interfaz de terminal se

corrompía con salidas de registro dirigidas a la salida estándar, y el registro debía dirigirse a la salida de error de forma consistente.

10.3 Bibliotecas de Python

Biblioteca	Uso
<code>maturin</code>	Construcción de la extensión nativa que expone el sistema a Python
<code>numpy</code>	Lectura de los conjuntos de datos binarios en la evaluación
<code>matplotlib</code>	Generación de todas las figuras de los experimentos
<code>safetensors</code>	Carga de los pesos entrenados por el sistema para su evaluación
<code>torch</code>	Línea de base de referencia
<code>ruff</code>	Análisis estático y formato

El uso de **PyTorch** requiere una aclaración explícita, porque podría malinterpretarse dado que el trabajo declara no usar frameworks de aprendizaje profundo. PyTorch **no participa del sistema en ninguna forma**: no se utiliza para entrenar, ni para calcular gradientes, ni para representar modelos. Se lo emplea exclusivamente como **línea de base externa**, entrenando en un solo proceso el mismo modelo con la misma receta de hiperparámetros, para tener un punto de referencia contra el cual contrastar la exactitud que alcanza el motor propio. Es un instrumento de medición, y su presencia en el repositorio es una dependencia opcional del entorno de experimentación, separada de las dependencias de ejecución del sistema.

10.4 Documentación y edición del informe

El presente informe y la propuesta que lo precedió se escriben en Markdown y se componen a PDF mediante **Pandoc**, con procesamiento de citas y bibliografía en formato BibTeX bajo el estilo APA 6. Las monografías que dieron origen a los estudios experimentales se compusieron en LaTeX con la clase de artículo de IEEE. Los diagramas se generaron con **Mermaid** y **Matplotlib**.

10.5 Herramientas de inteligencia artificial generativa

En cumplimiento de la obligación de declararlo explícitamente, se deja constancia de que durante el desarrollo del trabajo se utilizaron **modelos grandes de lenguaje** como asistencia, en los siguientes usos: consulta y discusión de conceptos de sistemas distribuidos y de aprendizaje profundo, asistencia en la depuración de errores, revisión y sugerencia de mejoras sobre código ya escrito, generación de código auxiliar en tareas accesorias como scripts de despliegue y de graficado, y asistencia en la redacción y revisión de la documentación y de este informe.

Los integrantes del equipo asumen la responsabilidad plena sobre la totalidad del contenido generado, se apropián del mismo, lo conocen en profundidad y están en condiciones de defenderlo en el acto de

Defensa. Delegar la ejecución de una tarea en una herramienta no exime de la responsabilidad sobre su resultado. En particular, se deja constancia de que las decisiones de arquitectura documentadas en este informe, los criterios metodológicos de los estudios experimentales y la interpretación de sus resultados son producto del trabajo y del debate del equipo, y su registro en las *issues* y *pull requests* del repositorio es verificable de forma independiente.

10.6 Consideración sobre licencias

La totalidad de las herramientas y bibliotecas utilizadas son de código abierto, con licencias permisivas (MIT o Apache 2.0 en su mayoría). El sistema desarrollado se publica bajo licencia abierta, de modo que pueda ser retomado y extendido por terceros, según el criterio establecido en la propuesta. No se incurrió en costos de licenciamiento, y el trabajo se desarrolló sobre hardware propio de los integrantes.

11 Experimentación y validación

La validación del trabajo se apoyó en dos frentes complementarios. El primero es el de las pruebas automatizadas, que verifican que el sistema hace lo que dice hacer. El segundo, y el que da sentido al proyecto, es el de los **estudios experimentales**: dado que O.N.O. fue concebido como una base común sobre la cual comparar estrategias de distribución de forma controlada, la validación última del trabajo consiste en demostrar que esa base efectivamente produce evidencia comparable. Esta sección presenta ambos frentes, y dedica la mayor parte de su extensión al segundo.

Los estudios se organizan como tres investigaciones independientes, cada una con su propia pregunta, su metodología y sus conclusiones acotadas al entorno evaluado. Las tres comparten el mismo sistema, el mismo motor de redes neuronales y el mismo plano de control: la única variable entre configuraciones es aquella que cada estudio se propone aislar. Esa es, precisamente, la propiedad que justifica haber construido el sistema desde cero en lugar de comparar implementaciones ajenas.

11.1 Pruebas automatizadas

El sistema se verificó con tres tipos de pruebas. Las **pruebas unitarias** ejercitan cada componente de forma aislada y son las que sostienen el motor de redes neuronales: la corrección de la propagación hacia atrás no es observable a simple vista, y la única manera práctica de detectar un gradiente mal derivado es contrastarlo contra un valor calculado de forma independiente. Las **pruebas de integración** validan la interacción entre nodos durante una ejecución distribuida, en particular los protocolos de coordinación. Las **pruebas de aceptación** comprueban que cada funcionalidad requerida se comporta como fue especificada, ejecutando un entrenamiento completo sobre el entorno simulado y verificando que la convergencia obtenida sea consistente con la del entrenamiento secuencial equivalente.

Las dos primeras se escribieron con el arnés de pruebas nativo de Rust y se ejecutan mediante **cargo test**; las de aceptación se levantan sobre los entornos contenedizados descritos más abajo. Las tres son automatizadas y de ejecución desatendida, y se versionan junto con el código, de modo que cada cambio pueda validarse antes de integrarse al trabajo.

Corresponde señalar una desviación respecto de lo planificado en la propuesta. Las pruebas de integración de más alto nivel, aquellas que levantaban un clúster completo desde Python, fueron **abandonadas y reemplazadas** por la suite de benchmarks. La razón es que nunca se acordó un estándar de repositorio para las configuraciones de Python y Docker que esas pruebas requirieran, de modo que su mantenimiento resultaba más costoso que la confianza que aportaban; la suite de benchmarks, que ejecuta entrenamientos completos y verifica sus métricas, cubre el mismo terreno con un propósito adicional. Este punto se retoma en *Riesgos materializados y lecciones aprendidas*.

11.2 Entorno de experimentación y reproducibilidad

Todas las mediciones se realizaron sobre clústeres **simulados con Docker** en una única máquina física: un contenedor por nodo, red *bridge* y puertos publicados. Esta decisión, tomada al inicio del trabajo por la imposibilidad de disponer de múltiples máquinas dedicadas, es la limitación transversal a los tres estudios y condiciona la lectura de todos los resultados de rendimiento. Al aumentar la cantidad de nodos, los núcleos físicos se saturan: lo que se mide es **escalado lógico bajo recursos**

compartidos, no escalado multimaquina. La red, además, es efectivamente loopback, sin latencia ni ancho de banda realistas, lo que **favorece sistemáticamente a las estrategias que comunican más**. Cada estudio explicita cómo esa limitación afecta a sus propias conclusiones.

Para que las comparaciones sean reproducibles, cada ejecución se documenta junto con la configuración que la generó, se fija la semilla de inicialización de los parámetros y se conservan tanto los resultados crudos como los procesados. Los conjuntos de datos utilizados se resumen en la Tabla 1.

Cuadro 4: Conjuntos de datos utilizados en la evaluación.

Dataset	Entrenamiento	Test	Forma	Clases
MNIST	60 000	10 000	$28 \times 28 \times 1$	10
FashionMNIST	60 000	10 000	$28 \times 28 \times 1$	10

MNIST (LeCun, Bottou, Bengio, & Haffner, 1998) se emplea como referencia de correctitud y convergencia, por tratarse de un problema bien caracterizado sobre el que cualquier desviación resulta evidente. FashionMNIST (Xiao, Rasul, & Vollgraf, 2017) se emplea como benchmark principal en el primer estudio, por ser una tarea comparable en dimensiones pero considerablemente menos separable, lo que evita que todas las configuraciones saturan en exactitudes indistinguibles.

11.3 Estudio I: Parameter Server frente a All-Reduce

11.3.1 Pregunta e hipótesis

El primer estudio aborda la pregunta que motivó el trabajo desde la propuesta: **¿cuándo conviene cada enfoque, por qué, y qué compromisos aparecen entre convergencia, throughput, escalabilidad y sincronización?** No se busca aquí el estado del arte en visión por computadora, sino caracterizar el *sistema* de entrenamiento. Es la pregunta que O.N.O. fue construido para responder, y por eso este estudio es el que valida más directamente la premisa del proyecto.

11.3.2 Fundamento

Con W workers, cada uno procesa un mini-batch local de tamaño b y calcula un gradiente g_w . El gradiente agregado es el promedio

$$\bar{g} = \frac{1}{W} \sum_{w=1}^W g_w,$$

que equivale a un paso de descenso por gradiente con **batch efectivo** $W \cdot b$, es decir, el tamaño de batch que efectivamente “ve” el optimizador tras agregar a los W workers. Promediar, y no sumar, mantiene la magnitud del gradiente, y por consiguiente la tasa de aprendizaje efectiva, independiente de W . Esta distinción es central para el diseño experimental: en O.N.O. el `batch_size` es **por worker**, de modo que, con b constante, agregar workers cambia el batch efectivo.

En el régimen de Parameter Server evaluado, los servidores almacenan los pesos fragmentados entre ellos, y los workers hacen *push* de gradientes y *pull* de parámetros. El régimen de sincronización define el comportamiento: con barrera, todos los workers esperan a que el servidor aplique la actualización. En ring All-Reduce, los W workers se ordenan en anillo y ejecutan dos fases, *scatter-reduce* y *all-gather*; el volumen de gradiente que comunica cada worker es aproximadamente

$$2 \frac{W-1}{W} |g|,$$

esencialmente independiente de W salvo un factor constante, lo que le confiere buena eficiencia de ancho de banda. No hay servidor central ni punto único de contención, pero todos avanzan al ritmo del más lento.

Este estudio utiliza el régimen **sincrónico con barrera** para Parameter Server. La consecuencia es deliberada y debe tenerse presente al leer los resultados: bajo ese régimen, ambas estrategias aplican **la misma regla de actualización**, de modo que no cabe esperar una diferencia sistemática de convergencia entre ellas. Lo que se compara, entonces, es el costo de llegar al mismo lugar.

11.3.3 Criterio de comparación justa

Comparar dos estrategias que no consumen los mismos recursos exige explicitar el criterio de equidad, porque distintos criterios conducen a conclusiones distintas. El criterio adoptado es **igual número de workers**: para cada N , All-Reduce usa N workers y Parameter Server usa los mismos N workers

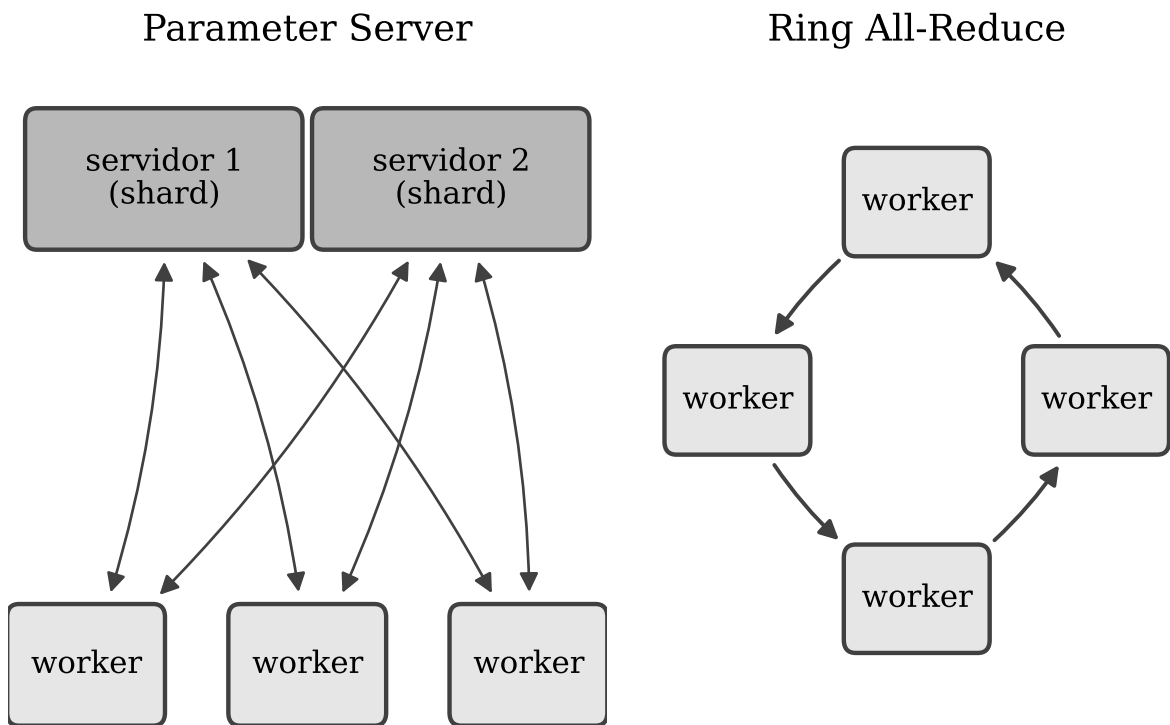


Figura 1: Parameter Server (izquierda): los workers hacen push y pull contra dos servidores que fragmentan los pesos. Ring All-Reduce (derecha): los workers promedian gradientes entre sí sin servidor central.

más dos servidores. Los servidores se consideran el costo estructural propio de Parameter Server, no un recorte de su capacidad de cómputo.

Este criterio tiene una consecuencia deseable. Como ambas estrategias usan los mismos W workers con el mismo b , el batch efectivo $W \cdot b$ es idéntico para ambas en cada topología. La semántica de optimización queda fijada y, por consiguiente, la convergencia es directamente comparable. El costo del criterio, que se declara abiertamente, es que Parameter Server ocupa más nodos totales para el mismo trabajo, lo cual queda registrado como una desventaja suya en la matriz de decisión final.

Se reportan dos mediciones complementarias, y se es explícito sobre qué pregunta responde cada una: con un **batch por worker pequeño** se mide convergencia y exactitud; con un **batch por worker mayor** se mide throughput bruto.

Cuadro 5: Topologías evaluadas bajo el criterio de igual número de workers. Parameter Server emplea dos servidores con parámetros fragmentados. Batch efectivo = workers $\times$ 10.

Estrategia	Workers	Servidores	Nodos	Batch efectivo
AR	3	0	3	30
AR	5	0	5	50
PS	3	2	5	30
PS	5	2	7	50

Los modelos empleados se detallan en la Tabla 3. El modelo principal es **nielsen**, una red convolucional pequeña con softmax y entropía cruzada. Para aislar la presión de comunicación se definieron además redes densas de tamaño creciente cuyo objetivo no es la exactitud sino variar el tamaño del gradiente.

Cuadro 6: Modelos empleados y su tamaño aproximado. Las redes Densa S/M/L son totalmente conectadas.

Modelo	Arquitectura	Parámetros
Nielsen	Conv20@5 $\rightarrow$ Pool $\rightarrow$ Dense100 $\rightarrow$ Dense10	289 630
Densa S	Dense128 $\rightarrow$ Dense10	101 770
Densa M	Dense512 $\rightarrow$ Dense512 $\rightarrow$ Dense10	669 706
Densa L	Dense1024 $\rightarrow$ Dense1024 $\rightarrow$ Dense10	1 863 690

Se utilizó el optimizador de descenso por gradiente estocástico con tasa de aprendizaje 0,1 y semilla fija, sobre una máquina de ocho núcleos.

11.3.4 Resultados: convergencia

Ambas estrategias entrenan correctamente. A igual número de workers, All-Reduce y Parameter Server alcanzan exactitudes de test prácticamente iguales en ambos conjuntos de datos, con Parameter

Server incluso marginalmente por encima en FashionMNIST. Las curvas de pérdida descenden de forma casi solapada.

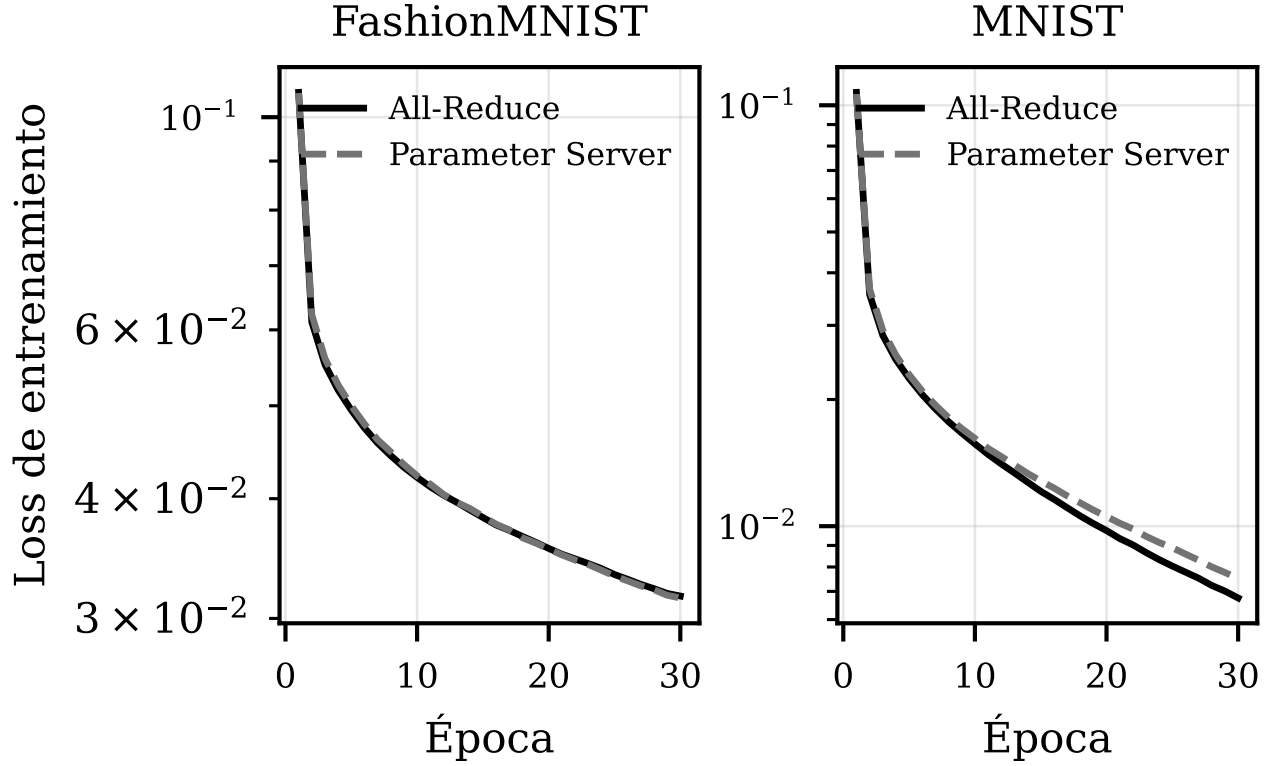


Figura 2: Convergencia (pérdida de entrenamiento por época) a 3 workers, en FashionMNIST (izquierda) y MNIST (derecha), para All-Reduce y Parameter Server.

Este resultado no es una sorpresa sino una **confirmación de corrección**: como el régimen es sincrónico, ambos esquemas de agregación aplican la misma regla de actualización, y una divergencia sistemática entre ellos habría indicado un error de implementación en alguno de los dos. En ese sentido, esta medición funciona como prueba de aceptación de ambas estrategias tanto como resultado experimental.

Cuadro 7: Resultados de convergencia justa (batch 10 por worker, evaluación sobre el conjunto de test completo) para ambas estrategias en cada conjunto de datos.

Dataset	Estrategia	Workers	Exactitud test (%)	Tiempo total (s)
FashionMNIST	AR	3	87,28	210,9
FashionMNIST	PS	3	87,58	257,6
FashionMNIST	AR	5	86,25	248,1
FashionMNIST	PS	5	86,49	272,5
MNIST	AR	3	97,82	213,8
MNIST	PS	3	97,62	251,5

Dataset	Estrategia	Workers	Exactitud test (%)	Tiempo total (s)
MNIST	AR	5	97,00	253,4
MNIST	PS	5	96,75	269,1

La diferencia decisiva, entonces, es el **tiempo**. All-Reduce alcanza la misma exactitud apreciablemente antes que Parameter Server, con una brecha del orden del 15 % a 20 % en las configuraciones de 3 workers. La explicación es estructural: el servidor de Parameter Server actúa como punto de serialización por el que pasan todas las actualizaciones.

11.3.5 Resultados: throughput y escalabilidad

Con batch por worker fijo, el comportamiento al sumar nodos revela un cruce que matiza la conclusión anterior. Con pocos workers, All-Reduce sostiene mayor throughput; al sumar workers, las tendencias se invierten: el throughput de All-Reduce cae levemente, porque su anillo sincrónico satura los núcleos físicos, mientras que el de Parameter Server crece y llega a igualarlo.

Parameter Server escala con pendiente positiva al sumar nodos, amortizando el costo fijo de sus servidores. Debe recordarse, sin embargo, que para hacerlo emplea dos nodos servidores adicionales que All-Reduce no necesita, y que la saturación de núcleos observada en All-Reduce es un artefacto del entorno de máquina única, no una propiedad del algoritmo.

11.3.6 Resultados: presión de comunicación

Para aislar el efecto del tamaño del gradiente se entrenaron las redes densas de tamaño creciente a 3 workers, con batch por worker fijo. El throughput se desploma al crecer el modelo: la comunicación, y no el cómputo, pasa a dominar. All-Reduce sostiene mayor throughput que Parameter Server en los tres tamaños, pero su ventaja se **reduce** al crecer el modelo, porque el fragmentado de parámetros entre los dos servidores reparte mejor la presión de comunicación.

Este comportamiento se entiende mejor con una cuenta independiente de la red. En All-Reduce, cada nodo comunica aproximadamente $2\frac{W-1}{W}|g|$, esencialmente constante; en Parameter Server, el tráfico que ingresa a cada servidor crece de forma lineal con el número de workers, atenuado por el fragmentado entre S servidores. Esta métrica analítica explica tanto la eficiencia de ancho de banda de All-Reduce como el porqué el servidor se vuelve el punto de presión al sumar workers, y lo hace **con independencia de la velocidad de la red subyacente**, que es justamente la variable que el entorno de máquina única no permite explorar.

La compresión de gradientes, dispersa o cuantizada, es una mitigación conocida para esta presión (Aji & Heafield, 2017; Lin et al., 2018). O.N.O. la implementa, y de hecho el segundo estudio la utiliza; su evaluación aislada quedó fuera del alcance.

11.3.7 Discusión y matriz de decisión

Cuándo conviene All-Reduce. La evidencia lo favorece en el escenario evaluado: clúster homogéneo, red rápida, entrenamiento sincrónico y gradientes densos. Iguala a Parameter Server en convergencia y entrega mayor throughput y menor tiempo hasta exactitud, sin un servidor central que congestione.

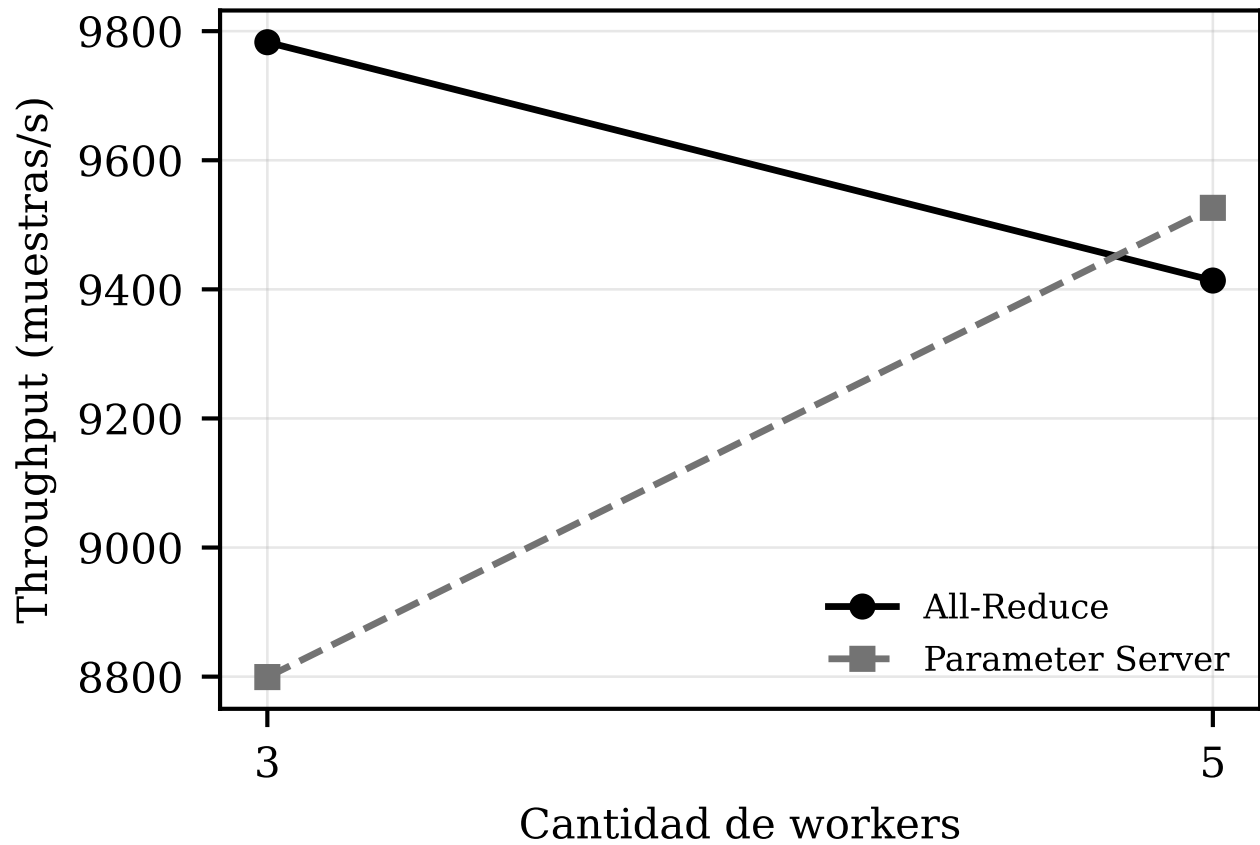


Figura 3: Throughput (muestras por segundo) frente al número de workers, con batch por worker fijo, en FashionMNIST. All-Reduce parte más alto pero Parameter Server lo alcanza al crecer los workers.

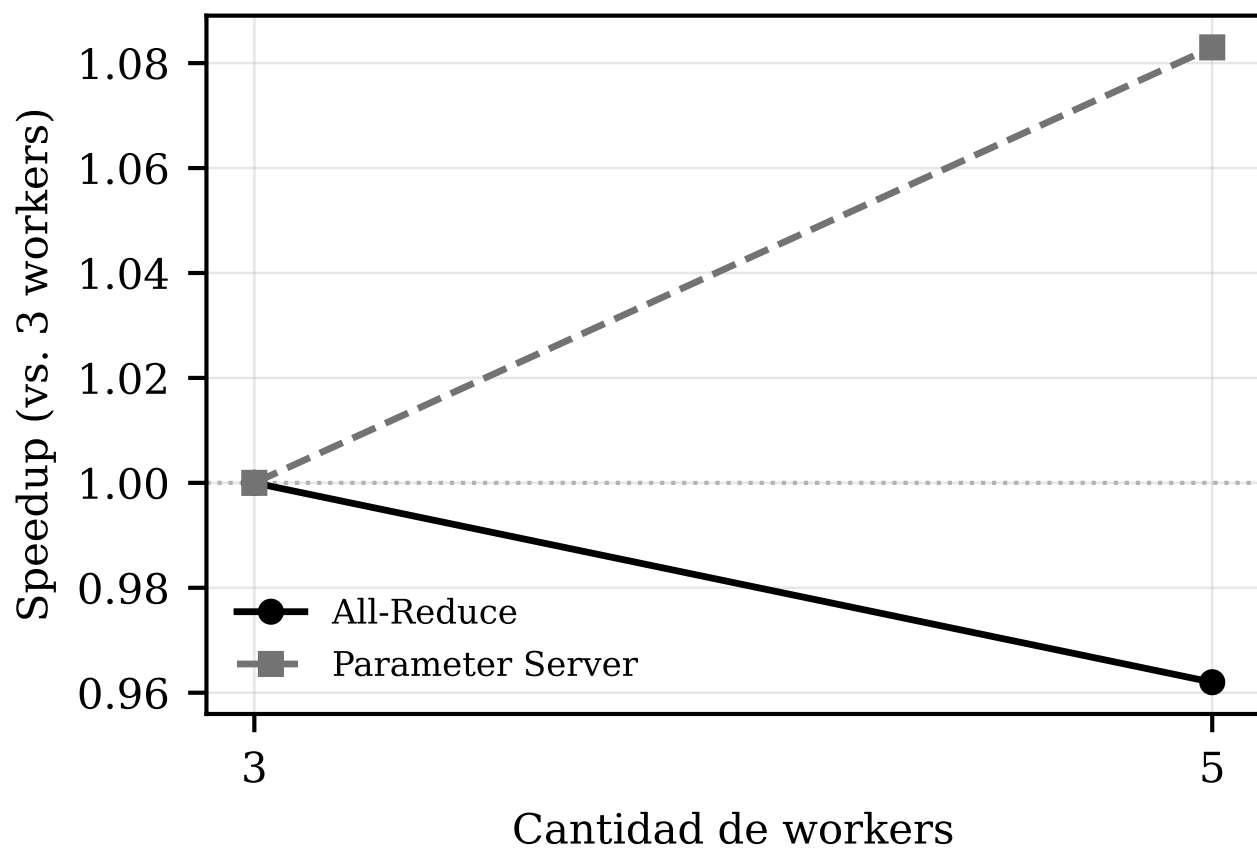


Figura 4: Speedup de throughput respecto de la configuración de 3 workers. Parameter Server escala con mayor pendiente que All-Reduce desde una base más baja.

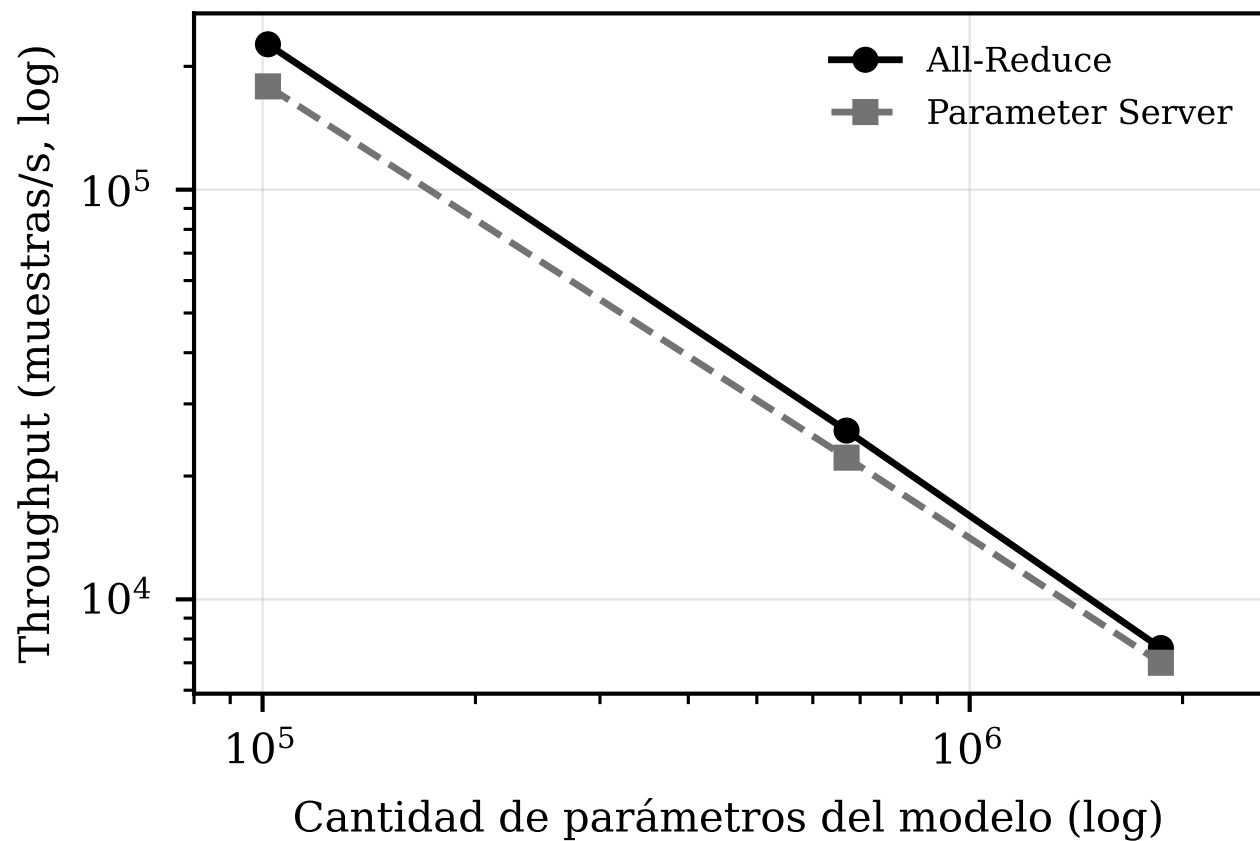


Figura 5: Presión de comunicación: throughput (escala logarítmica) frente al tamaño del modelo (escala logarítmica) a 3 workers. All-Reduce sostiene mayor throughput en los tres tamaños; la ventaja se reduce al crecer el modelo.

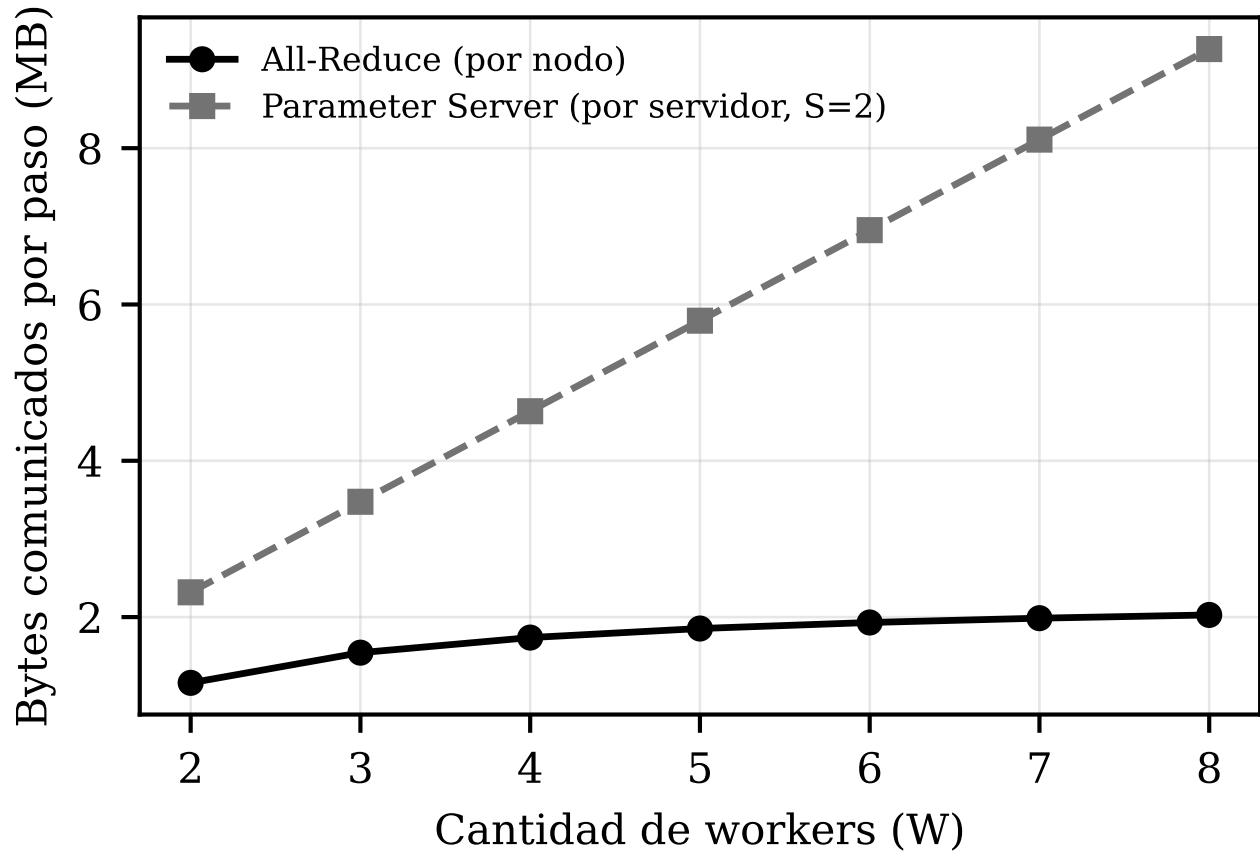


Figura 6: Volumen de datos comunicado por paso (métrica analítica, independiente de la red) frente al número de workers. En All-Reduce el volumen por nodo es casi constante; en Parameter Server el tráfico por servidor crece con los workers.

Cuando los datos y los nodos son homogéneos, y no hay por tanto heterogeneidad ni nodos rezagados que un esquema asincrónico pueda aprovechar, All-Reduce resulta la elección natural.

Cuándo conviene Parameter Server. La evidencia lo favorece en dos aspectos parciales: escala con mayor pendiente al sumar nodos, amortizando el costo fijo de sus servidores, y su fragmentado de parámetros empieza a acortar la brecha de throughput con los modelos más grandes. Su ventaja estructural más fuerte, el fragmentado de modelos que no entran en un solo nodo, sería decisiva a mayor escala, pero el modelo mayor evaluado sí completó en All-Reduce, por lo que no llegó a ejercitarse. El resto de sus ventajas, heterogeneidad, nodos rezagados y asincronía, no fueron evaluadas.

Cuadro 8: Matriz de decisión cualitativa All-Reduce frente a Parameter Server, acotada al entorno evaluado.

Criterio	All-Reduce	Parameter Server
Convergencia	Empate: misma semántica síncrona.	Empate: misma semántica síncrona.
Throughput / tiempo hasta exactitud	Favorable: sin servidor central.	Menor: sobrecarga del servidor.
Escalado al sumar nodos	Escala bien, pendiente base.	Amortiza su costo fijo: pendiente algo mayor.
Eficiencia de hardware	No requiere nodos servidores extra.	Requiere nodos servidores dedicados.
Presión de comunicación / modelos grandes	Sostiene la ventaja.	El fragmentado acorta la brecha.
Heterogeneidad / rezagados / asincronía	Síncrono: sensible a rezagados.	Ventaja estructural; no ejercitada aquí.

11.3.8 Limitaciones del estudio

El clúster se simula en una sola máquina de ocho núcleos: el escalado es lógico, no físico, y la red es loopback. Esta es la limitación principal y la que más pesa sobre la comparación de rendimiento, porque una red rápida **atenúa el costo de comunicación de Parameter Server**; de ahí que la métrica analítica de volumen comunicado deba leerse como complemento necesario de los tiempos medidos, y no como una ilustración accesorio. El estudio se restringe además al régimen sincrónico y a datos distribuidos de forma homogénea. O.N.O. es un sistema académico y no compite con frameworks industriales como Horovod (Sergeev & Del Balso, 2018) o BytePS (Jiang et al., 2020), que se usan aquí solo como marco conceptual.

11.3.9 Conclusión del estudio

En el entorno evaluado, All-Reduce resultó la opción más conveniente: converge a la misma exactitud que Parameter Server y lo supera en throughput y en tiempo hasta exactitud, debido a que el servidor central introduce un cuello de serialización. Esta ventaja, sin embargo, está atada a dichas condiciones: cuál de las dos estrategias conviene depende del contexto, y no constituye una preferencia absoluta. Parameter Server recorta distancia al escalar y con modelos grandes gracias al fragmentado,

pero no superó a All-Reduce en throughput aun con el modelo más grande; podría considerarse una elección natural solo si se explotaran sus ventajas estructurales, que aquí no fueron ejercitadas.

11.4 Estudio II: impacto de las épocas offline

11.4.1 Pregunta e hipótesis

Permitir que los nodos operen de forma autónoma durante varias épocas posterga el intercambio de parámetros, disminuyendo la dependencia de la red y priorizando el cómputo local. Sin embargo, demorar la sincronización introduce divergencia entre los pesos de los nodos, un fenómeno conocido como *client drift* (McMahan et al., 2017). O.N.O. expone este mecanismo como un parámetro de configuración, las **épocas offline** (E), lo que permite estudiarlo de forma directa.

La pregunta es: **¿cómo impactan las épocas offline en la velocidad y en la eficacia del modelo entrenado?** La hipótesis de partida era la que motiva la técnica: que un E mayor reduciría el tiempo total a costa de alguna pérdida de exactitud, y que existiría un punto de equilibrio aprovechable.

11.4.2 Metodología

Se ejecutaron entrenamientos sobre una misma arquitectura, conjunto de datos e hiperparámetros, variando únicamente el algoritmo distribuido, la cantidad de nodos y el valor de E . Todos los ensayos se realizaron de forma secuencial sobre la misma máquina, sumando un total de 10,5 horas de ejecución.

Cuadro 9: Configuración de los ensayos. En Parameter Server, la mitad de los nodos son workers y la otra mitad servidores.

Categoría	Parámetro	Valores
Modelo y datos	Dataset	MNIST completo (LeCun et al., 1998)
Optimización	Arquitectura	LeNet-5
	Función de pérdida	Entropía cruzada
	Optimizador	Adam ($\alpha = 0,01$; $\beta_1 = 0,9$; $\beta_2 = 0,999$; $\epsilon = 10^{-8}$)
	Tamaño de mini-batch	64
Infraestructura	Épocas máximas	48
	Serialización	Dispersa ($r = 0,5$)
Variables libres	Algoritmo	All-Reduce, Parameter Server sincrónico
	Nodos	2, 4, 6, 8, 10
	Épocas offline	0, 1, 2, 3, 4

Al parametrizar $E > 0$, los nodos entrenan aislados durante múltiples ciclos y el tráfico de red se reduce de forma aproximadamente inversamente proporcional a E . Al alcanzar la etapa de sincronización, la consolidación se realiza por promedio directo de los gradientes parciales, de modo que bajo descenso por gradiente la actualización responde a

$$W_{t+1} = W_t - \frac{\lambda}{N} \sum_{i=1}^N G_i.$$

Los ensayos se realizaron con la **serialización dispersa** que implementa el sistema, siguiendo el enfoque de Aji y Heafield (Aji & Heafield, 2017). El parámetro r acota el muestreo del gradiente que se transmite: se calcula la cardinalidad efectiva del mensaje como

$$k = \text{round}(|g| \cdot (1 - r)),$$

y ese valor k se utiliza para escoger los k elementos de mayor magnitud absoluta del tensor de gradientes local, fijando un umbral mínimo. Los elementos descartados se acumulan en un gradiente residual que se considera en el mensaje siguiente. Con $r = 0,5$, la mitad de los componentes queda fuera de cada mensaje.

El entorno de ejecución fue un procesador Intel Core i5-8350U (4 núcleos físicos, 8 hilos lógicos, 1,7 GHz base y 3,6 GHz turbo) con 8 GB de memoria DDR4, sobre Ubuntu 24.04 LTS, Docker 29.6.1, Rust 1.96.1 y CPython 3.14.0.

11.4.3 Resultados: tiempo de ejecución

La variación de E **no indujo mejoras significativas** en los tiempos de ejecución: las curvas de cada configuración se solapan de forma estricta en todos los ensayos. Dado que el sistema se ejecuta en una única máquina, la supresión del costo de comunicación no alcanza a compensar la sobrecarga de cómputo, contrariamente a la expectativa inicial.

La escalabilidad del rendimiento se estanca al superar los 4 nodos, y en All-Reduce la elevación a 10 nodos degrada el desempeño global por la alta contención de CPU.

En Parameter Server, en cambio, la configuración de 10 nodos (5 workers y 5 servidores) reduce el tiempo respecto de configuraciones menores. La explicación es contraintuitiva pero consistente con el entorno: mientras que en All-Reduce los procesos operan en cómputo continuo, el esquema sincrónico de Parameter Server introduce estados de inactividad obligatorios mediante barreras, y esa inactividad **disminuye la contención**, permitiendo una alternancia más eficiente de los hilos de ejecución sobre los cuatro núcleos físicos disponibles. All-Reduce resulta, no obstante, intrínsecamente más veloz en configuraciones de pocos nodos, por su distribución balanceada de responsabilidades sin dependencias centralizadas.

11.4.4 Resultados: exactitud

El impacto de E se manifiesta con mayor claridad en la exactitud, por ser una métrica independiente de las limitaciones del hardware de ejecución. En ambos algoritmos, la exactitud final decrece de forma **monótona** al incrementar E . La introducción de una única época offline ($E = 1$), que reduce la frecuencia de sincronización global a la mitad, ya provoca un deterioro inmediato de aproximadamente un punto porcentual.

La degradación se acentúa de forma proporcional al número de nodos. Al incrementar las entidades distribuidas, las particiones locales del conjunto de datos se reducen y se vuelven potencialmente

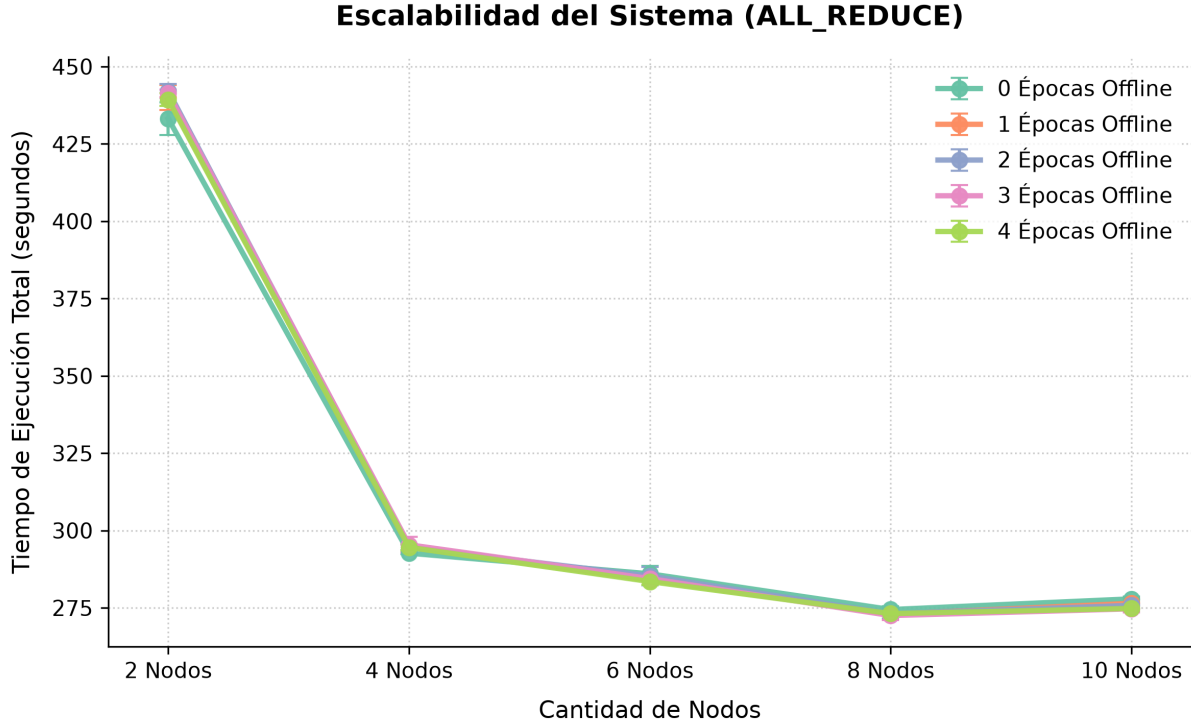


Figura 7: Comparación de tiempos de ejecución bajo distintas cantidades de épocas offline con All-Reduce.

sesgadas; la falta de sincronización frecuente exacerba el impacto de esos sesgos locales, perjudicando la convergencia global.

En All-Reduce la pérdida de exactitud es más severa que en Parameter Server. Esto se explica por el diseño experimental: bajo las condiciones evaluadas, All-Reduce duplica el número de workers activos al prescindir de servidores dedicados, lo que fragmenta aún más el conjunto de datos e incrementa el aislamiento operativo. La integración tardía de parámetros fuerza entonces correcciones abruptas del gradiente global.

11.4.5 Resultados: inestabilidad de la pérdida

A partir del historial de entrenamiento se seleccionaron trayectorias representativas que confirman que el incremento de E introduce inestabilidad en la función de pérdida, evidenciada mediante oscilaciones y picos proporcionales al valor de E .

La dinámica del optimizador presenta mayores dificultades en las épocas iniciales, la fase asociada a las correcciones de mayor magnitud, y luego se estabiliza al aproximarse a un mínimo local. Sin embargo, los valores de convergencia residual son sistemáticamente más altos a mayor E . El fenómeno sugiere que el desacoplamiento prolongado restringe la capacidad del optimizador para alcanzar el mínimo, induciendo un comportamiento análogo al de una tasa de aprendizaje excesivamente alta.

En All-Reduce se observa un patrón similar, aunque con oscilaciones relativamente más atenuadas, lo que **revierte la hipótesis inicial** que preveía mayor inestabilidad en el esquema descentralizado.

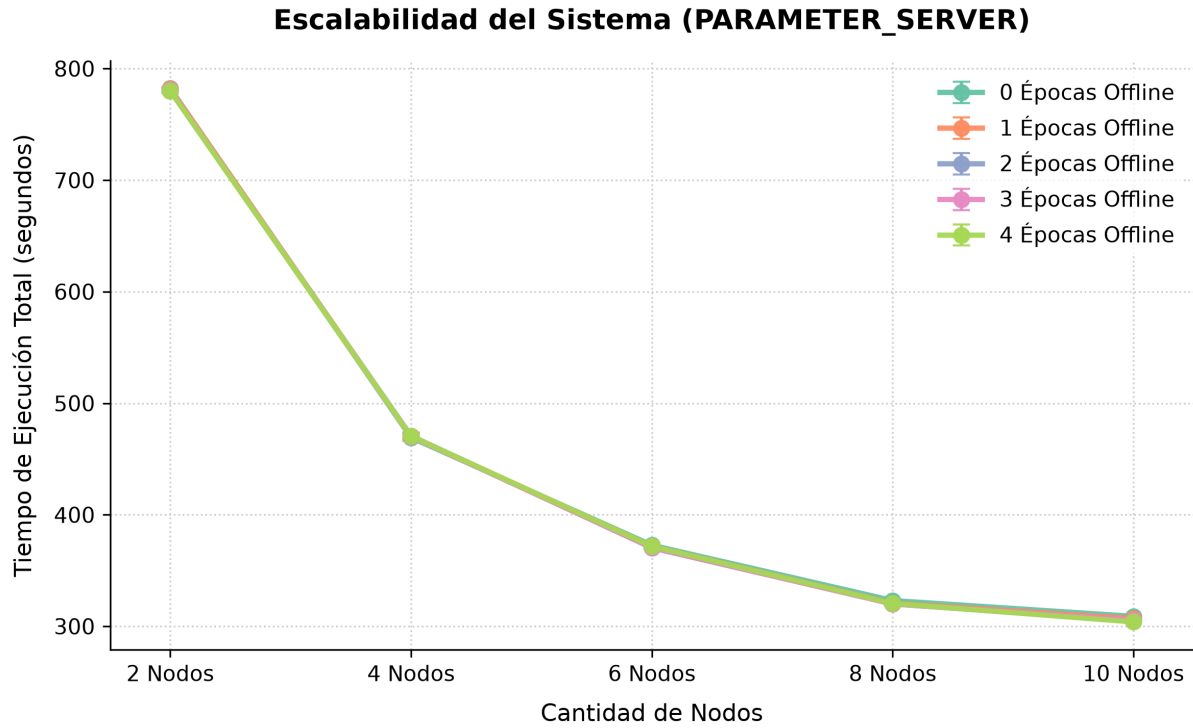


Figura 8: Comparación de tiempos de ejecución bajo distintas cantidades de épocas offline con Parameter Server.

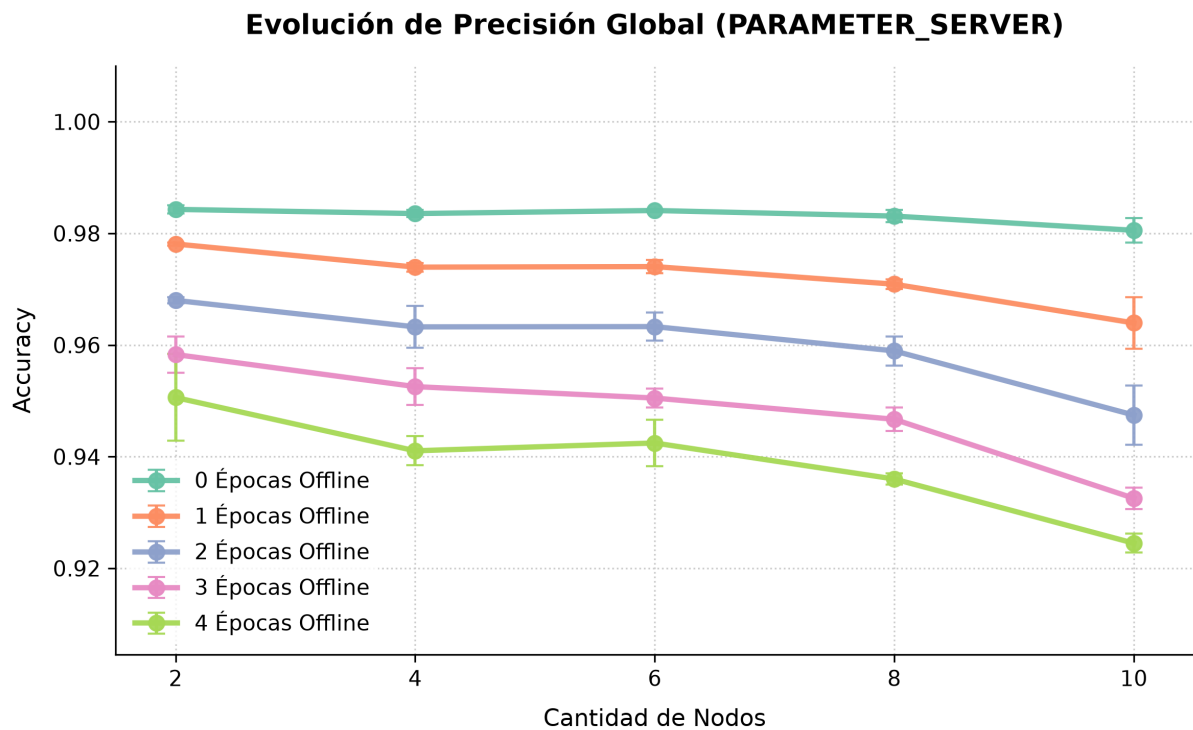


Figura 9: Comparación de exactitud bajo distintas cantidades de épocas offline con Parameter Server.

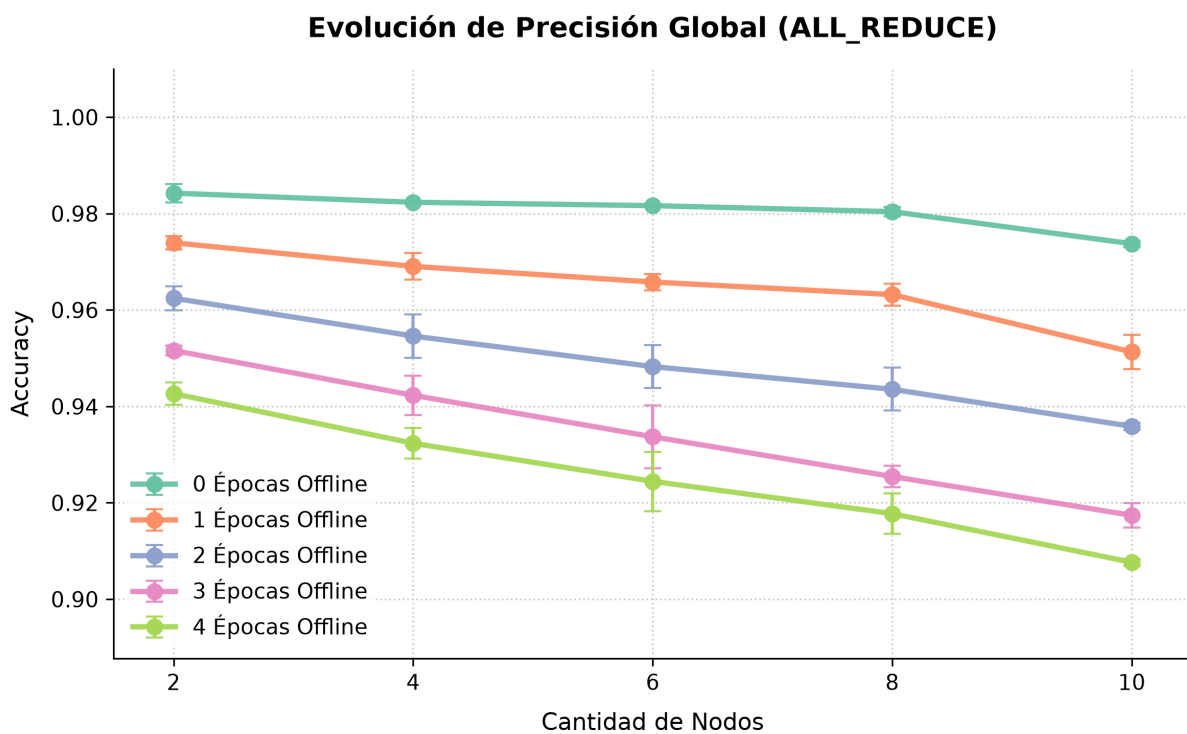


Figura 10: Comparación de exactitud bajo distintas cantidades de épocas offline con All-Reduce.

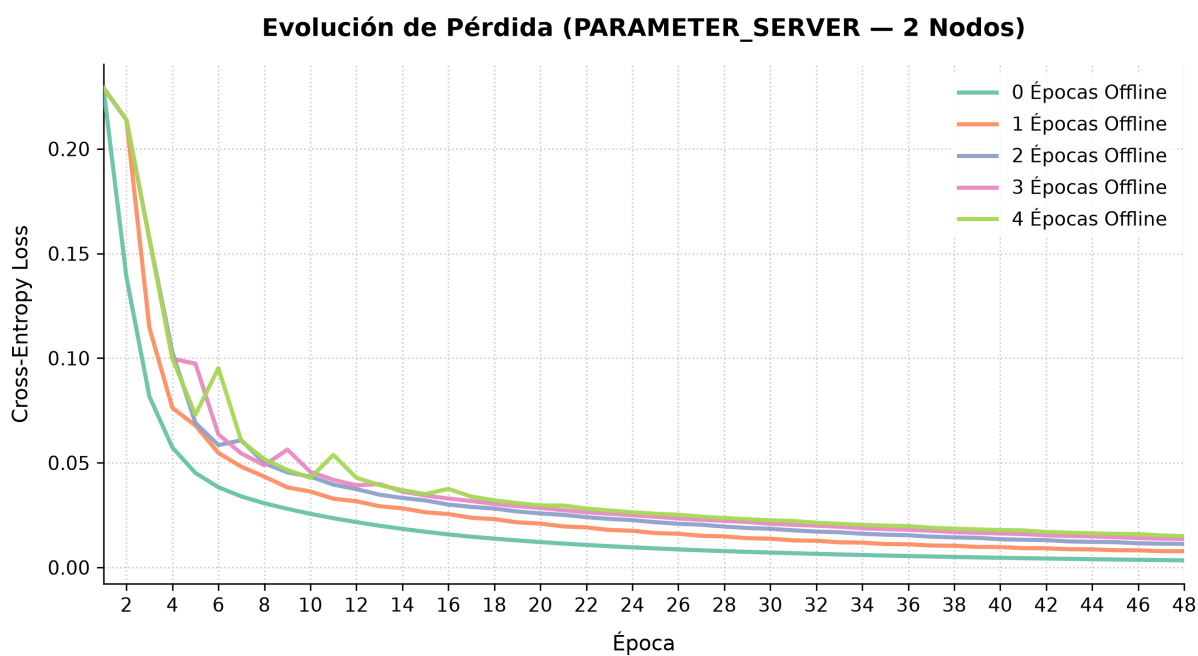


Figura 11: Trayectoria de pérdida con entropía cruzada exhibiendo ruido estocástico por cómputo offline en Parameter Server (2 nodos).

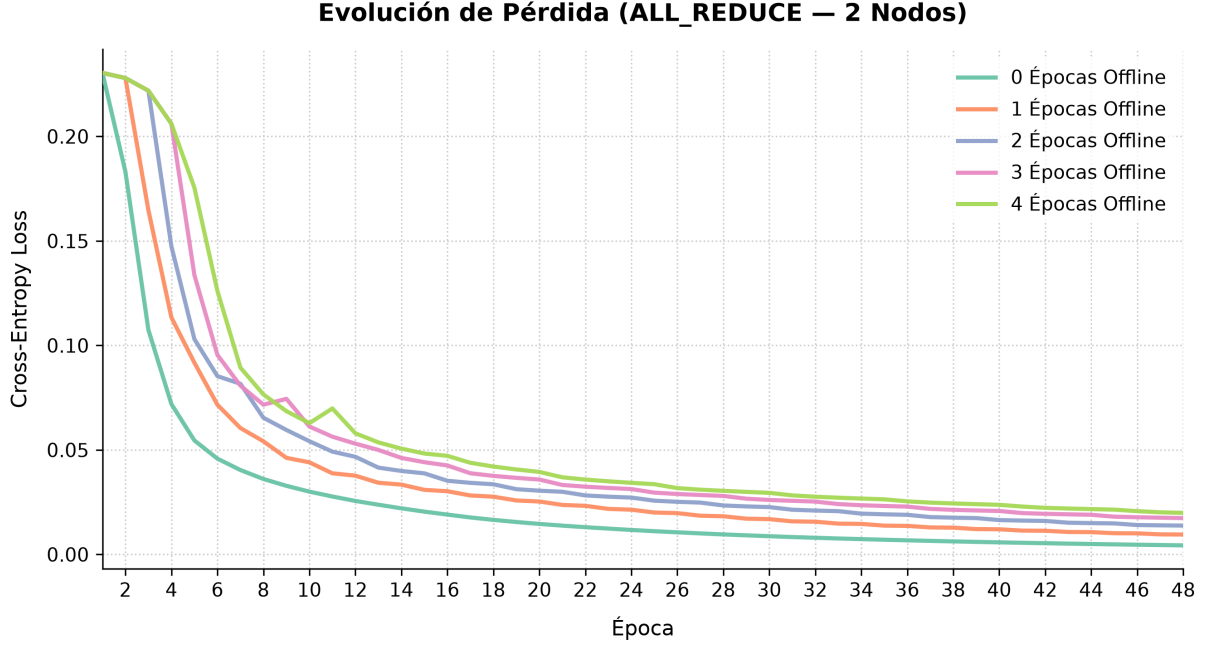


Figura 12: Trayectoria de pérdida con entropía cruzada exhibiendo ruido estocástico por cómputo offline en All-Reduce (2 nodos).

11.4.6 Discusión y limitaciones

La ventaja de introducir épocas offline es estrictamente reducir el tiempo de entrenamiento. En un entorno local simulado, eliminar el costo de comunicación no compensa el costo de obtener un peor modelo: en estos experimentos no se evidenció ninguna ventaja de utilizar un E mayor a 0.

En contraposición, en un despliegue multimáquina sobre una red física, retrasar la agregación de gradientes puede resultar crítico para amortizar el tiempo de comunicación. La decisión depende de tres factores: el tamaño del modelo, la velocidad de la red y la cantidad de nodos. Con modelos más grandes, los mensajes que contienen gradientes y parámetros son también más grandes. Para entrenamientos con mucho dato y poco modelo, el cuello de botella es el cómputo de gradientes; en cambio, para entrenamientos con poco dato y mucho modelo, donde la comunicación toma protagonismo, incrementar E puede ser útil para reducir los tiempos de ejecución.

Un resultado del estudio que merece señalarse es que la desestabilización afecta negativamente al modelo de forma predecible, lo que abre una línea concreta: evaluar estrategias de **activación dinámica** de las épocas offline, introduciéndolas exclusivamente en fases avanzadas del entrenamiento, cuando la optimización principal ha concluido, de modo que la exploración independiente de un worker en torno a un mínimo local pueda guiar favorablemente al resto tras la sincronización.

11.4.7 Conclusión del estudio

El uso de épocas offline en O.N.O. altera el balance entre comunicación y fidelidad algorítmica. En la infraestructura simulada, incrementar E perjudicó al modelo entrenado de manera predecible pero sin aportar ganancias de velocidad, debido a las limitaciones del hardware empleado. Queda

planteado validar el sistema bajo configuraciones multimáquina reales, comparando arquitecturas de modelos de distintos tamaños al variar E .

11.5 Estudio III: Strategy Switch

Sección pendiente de completar. Este estudio corresponde a la tercera investigación experimental del trabajo, centrada en el algoritmo *Strategy-Switch*, y será redactado por Alejo Ordoñez a partir de la monografía correspondiente. La estructura a seguir replica la de los dos estudios anteriores, de modo que los tres resulten comparables entre sí. Se deja aquí el esqueleto acordado.

11.5.1 Pregunta e hipótesis

[*Pendiente*] Formulación de la pregunta de investigación en torno a *Strategy-Switch* (Provatas et al., 2025): en qué medida iniciar el entrenamiento en régimen sincrónico de All-Reduce y promover workers a servidores de parámetros una vez que los gradientes se estabilizan permite combinar la exactitud del régimen sincrónico con la reducción de tiempo del régimen asincrónico.

11.5.2 Fundamento

[*Pendiente*] Descripción del criterio de conmutación implementado en O.N.O., de la regla empírica que dispara la transición y del mecanismo de promoción de workers a servidores, con referencia a las decisiones de diseño documentadas en *Solución implementada*.

11.5.3 Metodología

[*Pendiente*] Configuraciones evaluadas, conjuntos de datos, modelos, criterio de comparación justa frente a All-Reduce y Parameter Server puros, y entorno de ejecución.

11.5.4 Resultados

[*Pendiente*] Convergencia, tiempo hasta exactitud, momento efectivo de la conmutación y comportamiento posterior a la misma.

11.5.5 Discusión, limitaciones y conclusión del estudio

[*Pendiente*] Contraste con los dos estudios anteriores y con los resultados reportados en la literatura, y acotación de las conclusiones al entorno evaluado.

11.6 Síntesis de la validación

Los tres estudios responden preguntas distintas, pero convergen en una observación común sobre el entorno: **la simulación del clúster sobre una única máquina es la limitación que gobierna todos los resultados de rendimiento**. Favorece a las estrategias que comunican más, porque la red es loopback; penaliza a las que computan más, porque los núcleos se saturan; y hace que la contención de CPU aparezca como variable explicativa en lugares donde, sobre máquinas físicas separadas, no lo sería. Los resultados de **convergencia y exactitud**, en cambio, son independientes de esa limitación y pueden leerse con mayor confianza.

La segunda observación transversal es que el sistema cumplió su propósito. Que dos estrategias con arquitecturas de comunicación completamente distintas converjan a la misma exactitud bajo el mismo régimen de sincronización es, a la vez, el resultado experimental esperado y la mejor evidencia de que la base común es correcta: cualquier diferencia sistemática entre ellas habría delatado un error de implementación en alguna de las dos. Sobre esa base validada es que las diferencias de tiempo, throughput y escalabilidad admiten ser atribuidas al algoritmo y no a su implementación, que era exactamente el problema que este trabajo se propuso resolver.

12 Cronograma de las actividades realizadas

El plan de actividades de la propuesta fijó dieciséis tareas repartidas en cuatro hitos sobre dos cuatrimestres, con una carga estimada de 1500 horas de equipo. Esta sección contrasta ese plan con la ejecución real, que se extendió entre agosto de 2025 y julio de 2026.

12.1 Fases efectivamente recorridas

El desarrollo puede reconstruirse en once fases. Las fechas son aproximadas y corresponden a rangos de actividad, ya que varias fases se solaparon.

Cuadro 10: Fases del desarrollo efectivamente recorridas.

Fase	Período	Contenido
0	Ago–Nov 2025	Propuesta, relevamiento bibliográfico y un primer diseño descartado
1	Nov 2025–Ene 2026	Capa de comunicación
2	Dic 2025–Feb 2026	Parameter Server y Worker
3	Ene–Mar 2026	Motor de redes neuronales
4	Feb–Mar 2026	Orchestrator, interfaz de terminal e interfaz de Python
5	Mar 2026	Multi-servidor, conjuntos de datos y eficiencia de red
6	Mar–May 2026	All-Reduce en anillo y unificación del nodo
7	Abr–May 2026	Early stopping y la saga de bloqueos
8	May–Jun 2026	Strategy Switch, topología y benchmarks
9	Jun 2026	Endurecimiento y exposición de funcionalidad
10	Jun–Jul 2026	Consolidación del benchmark y auditoría de corrección
11	Jul 2026	Retracción de la tolerancia a fallos, optimización final y documentación

12.2 Contraste entre lo planificado y lo ejecutado

Lo que se cumplió según lo previsto. El núcleo del plan se completó: el sistema base parametrizable en Rust con su biblioteca de redes neuronales propia (tarea 4), los tres algoritmos (tareas 5 a 7), las optimizaciones de comunicación y sincronización (tareas 8 y 9), las dos interfaces (tarea 10), las pruebas (tarea 11), la simulación sobre distintas configuraciones de nodos (tarea 12) y el análisis de resultados con sus gráficos (tarea 14).

Lo que llevó más tiempo del estimado. Tres bloques se desviaron de forma significativa. El primero es la **fase 0**: casi cinco meses dedicados a documentación, relevamiento y trámites antes de escribir código productivo, con un primer diseño de arquitectura descartado por completo. El segundo es el **motor de redes neuronales** (tarea 4), cuya derivación manual de la propagación hacia atrás y la implementación de las capas convolucionales insumieron alrededor de dos meses de depuración de gradientes, muy por encima de lo previsto para un componente que la propuesta trataba como parte de una tarea mayor. El tercero es la **saga de bloqueos de coordinación** desatada por el early stopping (fase 7), que no figuraba en el plan bajo ninguna tarea y consumió un mes completo.

Lo que no se ejecutó. La tarea 13, optimización de carga de cómputo en configuraciones heterogéneas, con 150 horas estimadas, **no se abordó**: dependía de disponer de máquinas físicas con capacidades distintas, algo que el entorno de simulación sobre una única máquina no permite representar de forma convincente. La tolerancia a fallos, que la propuesta no listaba como tarea numerada pero que era parte del alcance declarado, se intentó tres veces sin éxito y finalmente se retrajo, según se detalla en *Riesgos materializados y lecciones aprendidas*.

Lo que se ejecutó sin estar planificado. Aparecieron cuatro líneas no previstas. La **selección de topología** mediante estadísticas de latencia y resolución del ordenamiento del anillo como un problema del viajante. La **auditoría de corrección distribuida** que surgió de la construcción de la suite de benchmarks y que destapó tres errores silenciosos de fondo. La **optimización del motor de convolución** mediante reformulación como multiplicación de matrices. Y las **dos monografías** que constituyen los estudios experimentales de este informe, que excedieron en profundidad lo que la tarea 14 contemplaba.

Una observación sobre la estimación. La distribución de horas de la propuesta asignaba 200 horas a investigar las implementaciones distribuidas de TensorFlow y PyTorch (tareas 2 y 3). En la práctica, ese estudio se realizó pero de forma mucho más acotada y en modalidad de consulta puntual, y el esfuerzo se redistribuyó hacia la implementación y la depuración del motor de aprendizaje. La lección de estimación es la habitual y no por conocida menos real: **el esfuerzo se subestima sistemáticamente en aquello que no se conoce de antemano**, que en este trabajo fue todo lo relativo a la corrección numérica del entrenamiento y a la coordinación distribuida.

12.3 Carga horaria efectiva

Cuadro 11: Contraste entre la carga horaria estimada en la propuesta y la efectivamente dedicada.

Nro.	Tarea	Horas estimadas	Horas reales
1	Leer y analizar los trabajos previos	100	[a completar]
2	Investigar la implementación distribuida de TensorFlow	50	[a completar]

Nro.	Tarea	Horas estimadas	Horas reales
3	Investigar la implementación distribuida de PyTorch	50	<i>[a completar]</i>
4	Desarrollar el sistema distribuido en Rust	150	<i>[a completar]</i>
5	Implementar Parameter Server	100	<i>[a completar]</i>
6	Implementar All-Reduce	100	<i>[a completar]</i>
7	Implementar Strategy-Switch	100	<i>[a completar]</i>
8	Optimizaciones de comunicación entre nodos	150	<i>[a completar]</i>
9	Optimizaciones de sincronización de las copias del modelo	150	<i>[a completar]</i>
10	Interfaz de funciones externas en Python e interfaz de terminal	50	<i>[a completar]</i>
11	Pruebas unitarias y de integración	50	<i>[a completar]</i>
12	Simulación sobre distintas configuraciones de nodos	100	<i>[a completar]</i>
13	Optimizaciones de carga en configuraciones heterogéneas	150	0 (no ejecutada)
14	Análisis de resultados y gráficos	100	<i>[a completar]</i>
15	Documentación del código y del proceso	50	<i>[a completar]</i>
16	Informe final	50	<i>[a completar]</i>
	Total	1500	<i>[a completar]</i>

13 Riesgos materializados y lecciones aprendidas

La propuesta identificó cuatro riesgos iniciales. Tres se materializaron, uno de ellos con una forma distinta de la prevista, y aparecieron otros que no estaban contemplados. Esta sección los recorre con la evidencia concreta del historial del proyecto, y cierra con las lecciones que dejaron. Se ha optado por documentar los desvíos con precisión antes que por presentar un recorrido más prolijo del que efectivamente ocurrió.

13.1 Riesgos previstos que se materializaron

La complejidad de la implementación distribuida en Rust. Se materializó, y fue el riesgo dominante del proyecto. Las garantías de *fearless concurrency* del lenguaje cumplieron lo que prometen, ya que ninguna condición de carrera llegó a producción por la vía habitual, pero desplazaron la dificultad hacia otro terreno: el de los **bloqueos por coordinación distribuida**, que el compilador no puede detectar porque no son un problema de memoria compartida sino de protocolo.

La saga más costosa fue la del *early stopping*. Implementarlo de punta a punta desató una cascada de bloqueos cuya raíz conceptual es siempre la misma: **membresía estática frente a terminación dinámica**. El contador de la barrera se fijaba en N workers al inicio; cuando un worker detenía su entrenamiento y se desconectaba, la barrera quedaba esperando indefinidamente a alguien que ya se había ido. El capítulo más didáctico de esa saga, sin embargo, llegó al final y tuvo otra naturaleza: el anillo de All-Reduce se bloqueaba con modelos grandes porque enviaba y recibía de forma **secuencial**, de modo que todos los nodos intentaban enviar antes de recibir y el buffer del socket se llenaba sin que nadie lo drenara. No era un problema de barreras ni de membresía, sino de control de flujo de TCP; se resolvió ejecutando envío y recepción de forma concurrente.

La disponibilidad de hardware para simulaciones representativas. Se materializó plenamente y es la limitación transversal del trabajo. La mitigación prevista, contenedores Docker con límites de recursos, se implementó, pero no alcanza a suplir lo que no está: al simular el clúster sobre una única máquina, la red es efectivamente loopback y los núcleos se saturan al sumar nodos. Esto tiene una consecuencia metodológica que atraviesa todos los resultados de rendimiento del informe, ya que **favorece sistemáticamente a las estrategias que comunican más y penaliza a las que computan más**. La respuesta que se dio, y que se considera la decisión metodológica más valiosa del trabajo, fue complementar cada medición de tiempo con una **métrica analítica de volumen de comunicación**, independiente de la velocidad de la red, de modo que el lector pueda separar lo que es propiedad del algoritmo de lo que es artefacto del entorno.

La amplitud del alcance y la estimación de tiempos. Se materializó. La mitigación prevista, priorizar un sistema base funcional y abordar las optimizaciones de forma incremental, se aplicó y funcionó; el costo fue que la línea de trabajo postergada nunca se cerró. Se retoma más abajo.

La dependencia de trabajos previos. Este riesgo se materializó de una forma que no se había anticipado. Lo previsto era que la interpretación de un algoritmo publicado pudiera diferir de lo documentado. Lo que efectivamente ocurrió fue más sutil y más interesante: **los supuestos de los papers no siempre se cumplen en el régimen propio**. Se detalla a continuación, porque es la lección técnica más importante del proyecto.

13.2 Riesgos no previstos

Aplicar un algoritmo sin verificar la validez de sus supuestos. El sistema implementa la actualización de parámetros libre de bloqueos siguiendo el trabajo de Niu et al. (Niu et al., 2011), detrás de un *trait* que permite elegirla en tiempo de ejecución. Una auditoría tardía mostró dos problemas independientes. El primero es de corrección del lenguaje: la implementación obtiene dos referencias mutables a la misma memoria desde múltiples hilos, lo cual en Rust es comportamiento indefinido. Esto no es equivalente a la condición de carrera “benigna” del trabajo original, escrito en C++, donde una carrera sobre memoria plana queda definida por la implementación; en Rust el compilador **asume que no ocurre** y optimiza en consecuencia. El síntoma no fue un fallo sino un cuelgue silencioso, y solo se manifestaba con varios workers concurrentes, razón por la cual las pruebas existentes, que ejercitaban únicamente la variante con bloqueo, nunca lo detectaron.

El segundo problema es conceptual y más profundo: *Hogwild!* converge bien con gradientes **ralos**, porque su argumento se apoya en que las colisiones entre actualizaciones concurrentes son infrecuentes. Una red convolucional densa, que es lo que O.N.O. entrena, produce gradientes densos en los que cada parámetro se actualiza en cada paso y las colisiones son masivas. El supuesto del paper, sencillamente, no se cumple en el régimen propio.

El análisis dejó al descubierto una tensión de diseño irreducible: **no se pueden tener a la vez un optimizador genérico con estado, una implementación realmente libre de bloqueos y corrección**. Usar operaciones atómicas daría una implementación correcta, pero solo con un optimizador sin estado, ya que el ciclo de lectura-modificación-escritura del estado interno de Adam o de momentum no admite ser libre de bloqueos; y además rompería el contrato del *trait* de optimizador, definido sobre referencias mutables. Poner un bloqueo por fragmento elimina el problema, pero entonces deja de ser libre de bloqueos y ya no es la técnica del paper. El debate quedó sin resolver y **la variante fue deshabilitada en la suite de benchmarks**. Se la reporta aquí como implementada pero no validada, y no como una de las variantes operativas del sistema.

Una decisión de arquitectura temprana que bloquea una funcionalidad tardía. Es el riesgo más instructivo del proyecto y merece detalle, porque no se trata de un error sino de un compromiso que solo se hizo visible dos fases después de haberlo asumido.

En la fase de construcción del worker se decidió representar el modelo como un único buffer plano con una vista computacional sin estado sobre desplazamientos y formas, de modo de permitir entrada y salida de red **sin copias**. Fue la decisión correcta y sostuvo el rendimiento del sistema durante todo el trabajo. Su consecuencia, sin embargo, es que el tipo de mensaje del protocolo **toma prestados** los bytes del buffer en lugar de poseerlos. Meses después, al intentar implementar la capa de reconexión que la tolerancia a fallos requería, ese préstamo resultó ser exactamente lo que lo impedía: un mensaje que toma prestada su memoria no puede moverse libremente entre tareas asincrónicas, y toda la capa de detección de caídas dependía de poder hacerlo. El análisis de alternativas enumeró cuatro diseños posibles y descartó los cuatro, cada uno por una razón distinta: mantener el socket dentro de la tarea de heartbeat era imposible por el préstamo; separar las mitades de lectura y escritura del socket le quitaba el propósito a la tarea; proteger la mitad de escritura con un mutex resolvía el problema pero introducía un olor de diseño evidente y un costo no cuantificado; y abrir un segundo socket duplicaba las conexiones y desacoplaba la detección de la caída del canal que efectivamente importaba.

La lección no es que la decisión original haya sido mala. Es que **las decisiones de arquitectura tienen un alcance temporal mayor que el de la funcionalidad para la que se toman**, y que el costo de revisarlas crece con la distancia. Cuando el compromiso se hizo visible, revertirlo implicaba tocar cinco crates.

El costo del refactor transversal como freno estructural. Ligado a lo anterior. El intento más completo de implementar reconexiones tocaba 41 archivos y cinco crates, llegó a integrarse y a probarse contra ambas estrategias, y aun así quedó congelado sin llegar a revisarse. Una queja registrada durante un refactor anterior de alcance similar, “*un quilombo*”, se repitió, y esta vez detuvo el trabajo. En un sistema donde una capa transversal atraviesa todos los componentes, el tamaño del cambio se vuelve por sí mismo una barrera.

Tres intentos de la misma funcionalidad sin una decisión previa de diseño. La tolerancia a fallos se intentó tres veces: una rama de reconexiones, una segunda que la tomó como base y le agregó heartbeats, y una tercera rehecha desde cero descartando el trabajo de las dos anteriores. Dos de ellas por autores distintos. El análisis de alternativas que finalmente identificó el obstáculo raíz llegó **después** de dos intentos de implementación. Es un error de proceso claro: el diseño debió preceder al código, y no al revés.

13.3 Desvíos de alcance

Corresponde declarar con precisión qué quedó fuera y por qué.

La tolerancia a fallos no se implementó. Fue diferida de forma consciente al comienzo del trabajo, con el criterio explícito de tener primero un producto mínimo funcionando, y luego los tres intentos descritos fallaron por la causa técnica ya explicada. Al cierre del proyecto se tomó la decisión inversa a la esperable: **desmontar la infraestructura preparatoria**. Las capas de reintento y de vencimiento que se habían construido meses antes con ese propósito fueron eliminadas, dejando la capa de comunicación reducida a un único nivel de enmarcado de mensajes.

La justificación es doble. Por un lado, los vencimientos **enmascaraban errores reales en lugar de mitigarlos**: un nodo ocupado procesando un modelo de casi dos millones de parámetros era declarado muerto por un vencimiento mal calibrado, y subir el umbral solo cambiaba “fallar rápido” por “colgarse”. Por otro, reintentar a nivel de transporte sobre una conexión TCP ya muerta no aporta nada sin un protocolo de reconexión que renegocie identidad y puerto, que es precisamente lo que no se logró construir. La semántica del sistema pasó a ser bloquear indefinidamente en lugar de fallar por vencimiento, lo cual es coherente con el modelo de fallos que el sistema efectivamente soporta, que es ninguno. Se prefirió un sistema declaradamente sin tolerancia a fallos antes que uno con una tolerancia aparente que no resiste el primer fallo real.

La optimización de carga en configuraciones heterogéneas no se abordó. Dependía de disponer de máquinas físicas con capacidades distintas, algo que el entorno de simulación no permite representar de forma convincente.

Las pruebas de integración de alto nivel fueron reemplazadas. Se explicó en la sección de experimentación: sin un estándar acordado para las configuraciones que requerían, su mantenimiento superaba la confianza que aportaban, y la suite de benchmarks cubrió el mismo terreno.

Un conjunto de líneas se cerró deliberadamente sin implementar. En una sesión de

definición de alcance previa al cierre se descartaron de forma explícita el multiplexado de sesiones, la estandarización del orden de bytes, el consumo de conjuntos de datos por flujo y la parametrización genérica del tipo de punto flotante. Se registran como descartes conscientes y no como pendientes.

13.4 Lecciones aprendidas

1. La instrumentación es una herramienta de detección de errores, no un anexo de medición. Esta es la lección central del trabajo. La suite de benchmarks se construyó para comparar estrategias y terminó destapando **tres errores de corrección del sistema distribuido** que llevaban meses en el código: que el anillo de All-Reduce **sumaba los gradientes en lugar de promediarlos**, de modo que la tasa de aprendizaje efectiva escalaba con la cantidad de workers; que los servidores de parámetros almacenaban las capas ordenadas por tamaño en lugar de por su orden en el modelo; y que al conmutar de estrategia los fragmentos de parámetros se asignaban siguiendo un orden distinto del que usaba el worker para indexarlos. Los tres eran **silenciosos**: no provocaban fallos, solo degradaban la exactitud. Ninguna prueba unitaria los habría encontrado, porque cada componente hacía correctamente lo que le correspondía.

Vale la pena detenerse en por qué el error del orden de capas sobrevivió tanto tiempo: **todas las redes de prueba tenían capas monótonamente decrecientes** (784 a 128 a 64 a 10), de modo que ordenar por tamaño coincidía por casualidad con el orden del modelo. Un conjunto de pruebas que no varía la forma de sus datos no puede distinguir una implementación correcta de una que acierta por accidente.

2. El paralelismo no siempre es la respuesta. La optimización del motor de aprendizaje se abordó al final del trabajo, con la intención declarada de “optimizar primero en un solo hilo y recién después ir por varios”. Se probaron tres enfoques para acelerar la convolución. La transformada rápida de Fourier se descartó. La paralelización con hilos se implementó y también se descartó, con una conclusión empírica clara: agregaba más sobrecarga que beneficio para el tamaño de problema evaluado. Lo que finalmente funcionó fue **reformular la convolución como una multiplicación de matrices**, para explotar las rutinas de álgebra lineal optimizadas y la localidad de caché. El resultado fue una mejora de 2,27 veces en el tiempo total, con la exactitud **subiendo** de 97,26 % a 98,5 %. La intuición inicial de agotar la optimización de un solo hilo antes de paralelizar resultó correcta, y el multithreading terminó siendo innecesario.

3. La duplicación de documentación produce divergencia igual que la del código. El esquema de configuración estaba replicado en tres archivos distintos y los tres habían divergido, describiendo nombres de campos que ya no existían. Se resolvió creando un documento canónico y eliminando las copias. Es la misma regla de única fuente de verdad que el proyecto había tenido que aplicar en el código, donde llegaron a convivir dos funciones de particionado distintas que solo coincidían en el caso de un único servidor.

4. Cerrar y reabrir una rama es una disciplina válida, pero cuesta de reconstruir. El equipo adoptó de forma sistemática el patrón de cerrar ramas de trabajo en curso y abrir una nueva y limpia en lugar de iterar sobre la existente. Ocurrió al menos seis veces en componentes centrales. Como práctica produce un historial de cambios legible y revisiones acotadas; su costo es que la trazabilidad del razonamiento queda repartida entre pull requests cerradas que un lector externo no encuentra si no las busca. Buena parte del esfuerzo de reconstrucción del proceso para este informe

se debió a eso.

5. Un bug abierto no es un fracaso; un bug cerrado sin diagnóstico sí es una deuda.

Un problema de gradientes que se desvanecían fue reportado, investigado y finalmente cerrado porque dejó de reproducirse, probablemente absorbido por el refactor de la convolución. Se lo declara aquí como **no reproducible y no como resuelto**, porque no hay un cambio al que atribuirle la corrección. La distinción importa: un problema que desapareció sin explicación puede volver.

6. Las garantías del lenguaje llegan hasta donde llega el modelo de memoria. Rust evitó, sin excepción, toda la clase de errores para la que fue diseñado. No evitó, ni podía evitar, los bloqueos de coordinación distribuida, los errores de derivación de gradientes ni los desalineamientos de protocolo entre dos nodos que cumplen individualmente su contrato. Elegir un lenguaje con garantías fuertes fue acertado, pero conviene tener claro cuál es exactamente el conjunto de problemas que esas garantías cubren, para no confiar en ellas fuera de él.

14 Impactos económico, social y ambiental del proyecto

La propuesta incluyó una evaluación preliminar de impacto, formulada con los conocimientos disponibles al inicio del trabajo. Corresponde ahora revisarla a la luz de lo efectivamente construido y medido, señalando tanto lo que se sostiene como lo que debe matizarse.

Impacto económico. El entrenamiento de modelos de aprendizaje profundo concentra buena parte de su costo en el tiempo de cómputo y en la infraestructura necesaria para sostenerlo. La premisa de que distribuir el entrenamiento reduce ese tiempo se verificó, pero los experimentos obligan a acotarla: la ganancia no es automática y depende de la relación entre el costo de cómputo y el costo de comunicación. Se observó que, al aumentar la cantidad de nodos por encima de la capacidad física disponible, el rendimiento se estanca e incluso se degrada; y que el volumen de comunicación crece con el tamaño del modelo hasta dominar el tiempo total. La conclusión económica honesta es que la distribución reduce el costo cuando la carga de cómputo por nodo justifica el costo de coordinarlos, y que un sistema como el desarrollado sirve justamente para determinar dónde está ese umbral antes de invertir en infraestructura.

Un segundo efecto económico, más directo, es que el trabajo se desarrolló íntegramente sobre hardware propio de los integrantes y con herramientas de código abierto, sin licencias ni servicios pagos. El costo del proyecto se reduce, por tanto, a las horas-persona invertidas. Esa misma propiedad se traslada al producto: O.N.O. no requiere aceleradores especializados ni infraestructura de nube para ser utilizado o extendido.

Impacto social. El trabajo se publica como base abierta y parametrizable, con el objetivo de acercar el entrenamiento distribuido a equipos que no cuentan con grandes clústeres y de ofrecer un punto de partida para la investigación y la docencia en el área. La disponibilidad de implementaciones de referencia de los tres algoritmos fundamentales sobre una misma infraestructura, junto con una interfaz accesible desde Python y una interfaz de terminal que permite observar el entrenamiento en vivo, facilita que otros retomen, comparen y extiendan el trabajo. El valor pedagógico resultó más concreto de lo previsto: al no delegar nada en un framework existente, el sistema expone la mecánica completa del entrenamiento distribuido, desde la propagación hacia atrás hasta el protocolo de red, y en ese sentido funciona como material de estudio además de como herramienta.

Corresponde también señalar el reverso. Toda mejora en la eficiencia del entrenamiento de modelos de aprendizaje profundo es una tecnología de doble uso: reduce la barrera de entrada tanto para aplicaciones beneficiosas como para aplicaciones que no lo son. Este trabajo no incorpora mecanismos de mitigación al respecto, y su escala es la de un sistema académico, pero la observación forma parte de una evaluación honesta.

Impacto ambiental. El tiempo de cómputo se traduce de forma directa en consumo energético, de modo que cualquier reducción del tiempo total de entrenamiento tiene un correlato en el consumo. Este es el punto en el que la evaluación preliminar requiere la corrección más fuerte. La distribución introduce un costo adicional de comunicación y, sobre todo, **replica el cómputo de coordinación en cada nodo**: un entrenamiento distribuido sobre N nodos consume más energía agregada que el mismo entrenamiento secuencial, aun cuando termine antes. Los experimentos realizados lo muestran con claridad, ya que el entrenamiento secuencial de referencia sobre MNIST alcanzó mayor exactitud en menos tiempo total que cualquiera de las configuraciones distribuidas evaluadas sobre una única máquina.

La distribución se justifica ambientalmente, entonces, cuando permite entrenar modelos que de otro modo no serían viables, o cuando el paralelismo se despliega sobre máquinas físicas independientes que de todas formas estarían encendidas. Las técnicas de reducción de tráfico implementadas en el sistema, la codificación de gradientes en media precisión y el protocolo de gradientes dispersos, apuntan precisamente a que la ejecución distribuida traduzca su ganancia de tiempo en un uso más eficiente de los recursos y no solo en un mayor consumo agregado. Cuantificar ese ahorro energético de forma directa excedió el alcance del trabajo y queda planteado como línea futura.

15 Trabajos futuros

El sistema quedó en un estado que admite ser retomado, y varias de las líneas que siguen tienen su diseño ya especificado en el repositorio. Se ordenan por relevancia.

Tolerancia a fallos. Es la frontera abierta más importante y la que el trabajo intentó cerrar sin lograrlo. El comportamiento esperado está completamente especificado por entidad: el orchestrator debe encolar los mensajes destinados a un par caído hasta que este se reconecte; el servidor de parámetros debe distinguir un servidor lento de uno muerto, sin declarar muertos a todos los workers por un vencimiento mal calibrado; y el worker de All-Reduce debe poder esperar la reconexión de sus vecinos en el anillo. Lo que falta es la pieza habilitante: una **capa de reconexión** en el transporte. Retomarla exige resolver primero el obstáculo de fondo documentado en este informe, que es que el tipo de mensaje del protocolo toma prestados sus bytes y por eso no puede moverse entre tareas asincrónicas. Cualquier intento futuro debería comenzar por decidir ese punto de diseño (mensaje con posesión de sus datos, o un mecanismo que evite tener que moverlo) **antes** de escribir código, que es exactamente el error de proceso que este trabajo cometió tres veces.

Un requisito adicional que ya fue relevado: dado que el sistema admite varias entidades corriendo sobre la misma dirección IP con puertos distintos, la reconexión necesita un pequeño protocolo por el cual, al establecerse una conexión, cada extremo anuncie el puerto en el que escuchará en caso de caerse y revivir.

Validación sobre un clúster físico real. Es la línea de mayor impacto sobre las conclusiones del informe. Todos los resultados de rendimiento están condicionados por haber simulado el clúster sobre una única máquina, con red loopback y núcleos compartidos. Repetir las tres campañas experimentales sobre máquinas físicas independientes, con latencia y ancho de banda reales, permitiría verificar cuáles de las conclusiones son propiedades de los algoritmos y cuáles artefactos del entorno. La predicción concreta que quedaría a prueba es que **la ventaja de All-Reduce sobre Parameter Server debería ampliarse**, dado que la red rápida del entorno simulado atenúa el costo de comunicación que penaliza al servidor central. En la misma línea, un despliegue real permitiría por fin ejercitar el escenario en el que las épocas offline tienen sentido, que es aquel en el que el costo de comunicación domina.

Caracterizar Strategy-Switch en un régimen que ejerce la conmutación. El algoritmo está implementado y conmuta efectivamente cuando la curva de pérdida se aplanan; lo que ocurrió es que, en la campaña de benchmarks ejecutada, el umbral de estabilidad tomado del trabajo original no llegó a alcanzarse dentro del presupuesto de épocas evaluado, de modo que esas corridas particulares quedaron registradas como sin conmutación. Queda pendiente caracterizar el algoritmo sobre configuraciones que sí la ejerciten, y ese es el objeto del tercer estudio experimental de este informe. En la misma línea, corresponde revisar la topología de la fase inicial de All-Reduce, que hoy se ejecuta sobre la totalidad de los nodos, incluidos los que luego serán promovidos a servidores, para que la comparación contra All-Reduce y Parameter Server puros se realice sobre la misma cantidad de trabajadores.

Resolver la actualización de parámetros libre de bloqueos, o retirarla. La variante inspirada en *Hogwild!* está deshabilitada por las razones expuestas. Hay dos caminos y conviene elegir uno de forma explícita. El primero es implementarla con operaciones atómicas, lo que da una versión correcta pero **solo admite un optimizador sin estado**, y por lo tanto obliga a revisar el contrato

del trait de optimizador. El segundo es usar un bloqueo por fragmento, que elimina el problema de corrección y conserva la semántica sin barrera, pero deja de ser libre de bloqueos. Cualquiera de los dos exige, antes, verificar empíricamente si el enfoque tiene sentido para modelos que producen **gradientes densos**, que es el régimen en el que el sistema opera y no aquel para el que el algoritmo fue concebido.

Multithreading del motor de aprendizaje. El trabajo optimizó la convolución reformulándola como multiplicación de matrices y descartó la paralelización con hilos porque, para el tamaño de problema evaluado, agregaba más sobrecarga que beneficio. Esa conclusión está acotada a MNIST y a modelos pequeños. Con modelos y lotes considerablemente mayores, el balance podría invertirse, y el trabajo dejó la medición hecha como línea de base contra la cual comparar.

Consumo de conjuntos de datos por flujo. El sistema carga las particiones del conjunto de datos en memoria. Un mecanismo de lectura por flujo desde disco permitiría trabajar con conjuntos que no entran en la memoria de un nodo, que es uno de los dos motivos originales por los que se distribuye un entrenamiento y el que este trabajo no llegó a ejercitar.

Ampliación del motor de redes neuronales. Quedan pendientes las convoluciones apiladas y una revisión de la función de pérdida de entropía cruzada para que valide o garantice que su entrada sea una distribución de probabilidad. Se conocen además dos defectos abiertos: la propagación hacia atrás del *max-pooling* pisa el gradiente cuando el paso es menor que el tamaño del filtro, y el script de descarga de MNIST crea un directorio donde debería crear un archivo.

Evaluación aislada de las técnicas de reducción de tráfico. El sistema implementa codificación de gradientes en media precisión y un protocolo de gradientes dispersos con acumulación del residuo. Ambas se utilizaron en los experimentos, pero **no se evaluaron de forma aislada**: no se midió el compromiso entre la tasa de compresión y la degradación de la convergencia. Es un estudio autocontenido, de alcance acotado y con toda la instrumentación ya disponible, que sería el candidato natural para una cuarta campaña experimental.

16 Conclusiones

El trabajo se propuso estudiar las principales estrategias de entrenamiento distribuido de modelos de aprendizaje profundo y construir un sistema que permitiera compararlas de forma controlada. Ese objetivo se cumplió.

Sobre el producto. Se construyó O.N.O., un sistema distribuido de entrenamiento escrito íntegramente en Rust, que incluye una biblioteca de redes neuronales desarrollada desde cero, una capa de comunicación con su propio protocolo, un plano de control y tres interfaces de uso. No se recurrió a ningún framework de aprendizaje profundo existente: la propagación hacia adelante y hacia atrás, los optimizadores, las funciones de pérdida y la representación de los tensores fueron implementados en el marco de este trabajo sobre una biblioteca que provee únicamente operaciones sobre arreglos numéricos. Sobre esa base se implementaron los tres algoritmos de referencia del área, se agregaron técnicas de reducción del tráfico de red y se instrumentó todo con una suite de experimentación reproducible.

Sobre los resultados. Los estudios experimentales muestran que, en un entorno homogéneo, sincrónico y simulado sobre una única máquina, All-Reduce y Parameter Server convergen a la misma exactitud, y que All-Reduce la alcanza en menos tiempo porque el servidor central introduce un punto de serialización; que la ventaja de All-Reduce se reduce al crecer el modelo, porque el fragmentado de parámetros reparte mejor la presión de comunicación; y que postergar la sincronización mediante épocas offline degrada la exactitud de forma monótona sin aportar, en ese entorno, la ganancia de tiempo que la motiva.

Corresponde señalar con claridad el alcance de estas conclusiones. Todas están acotadas a un clúster simulado sobre una sola máquina, con red loopback y núcleos compartidos. Esa limitación **favorece sistemáticamente a las estrategias que comunican más**, y por eso los resultados de rendimiento se presentaron siempre acompañados de una métrica analítica de volumen de comunicación, independiente de la velocidad de la red. Los resultados de convergencia y exactitud, en cambio, no dependen de esa limitación.

Sobre la innovación. El trabajo no innova en algoritmos: los tres implementados están publicados. Innova, si cabe el término, en el **instrumento**. La contribución concreta es una base de comparación de código abierto en la que las tres estrategias comparten la misma capa de comunicación, el mismo motor de redes neuronales y el mismo plano de control, de modo que la única variable entre dos ejecuciones sea la estrategia bajo estudio. Ese era el problema detectado en la propuesta, y la evidencia de que la base funciona es precisamente el resultado que menos llama la atención: que dos arquitecturas de comunicación completamente distintas converjan a la misma exactitud bajo el mismo régimen de sincronización. Cualquier diferencia sistemática entre ellas habría delatado un error de implementación.

Sobre lo que no se logró. La tolerancia a fallos, que era la línea de trabajo más ambiciosa del alcance original, no se implementó. Se intentó tres veces y se falló las tres, por una razón técnica precisa que este informe documenta: una decisión de arquitectura temprana, correcta y beneficiosa en su momento, resultó ser la que impedía construir la capa de reconexión. Al cierre se tomó la decisión de desmontar la infraestructura preparatoria antes que sostener una tolerancia a fallos aparente. Se considera que documentar con precisión por qué algo no se logró tiene más valor que presentarlo como una tarea pendiente sin explicación.

Sobre la formación. El trabajo obligó a atravesar de punta a punta tres dominios que la carrera aborda por separado. El de los **sistemas distribuidos**, donde la lección más dura fue que las garantías de un lenguaje seguro no alcanzan a los bloqueos de coordinación: el compilador de Rust evitó sin excepción toda la clase de errores para la que fue diseñado, y no evitó ni podía evitar que una barrera esperara indefinidamente a un nodo que ya se había ido, ni que un anillo se trabara por control de flujo de TCP. El del **aprendizaje profundo**, donde derivar y depurar la propagación hacia atrás a mano, sin la red de contención de la diferenciación automática, dejó una comprensión del entrenamiento que ningún framework habría permitido adquirir. Y el de la **ingeniería de software**, donde la lección central fue que la instrumentación no es un anexo de medición sino una herramienta de detección: la suite de benchmarks, construida para comparar estrategias, terminó destapando tres errores de corrección silenciosos que llevaban meses en el código y que ninguna prueba unitaria podía encontrar, porque cada componente hacía correctamente lo que le correspondía.

Se cierra el trabajo con un sistema funcionando, con evidencia experimental que lo respalda, con las limitaciones de esa evidencia declaradas y con la frontera que quedó abierta documentada al detalle suficiente para que otro la retome.

17 Aportes individuales

El desarrollo del trabajo se planteó como una responsabilidad compartida: las etapas de análisis, revisión de código y pruebas fueron llevadas adelante por los tres integrantes en todas las tareas. Sin perjuicio de ello, cada integrante concentró su foco principal en un conjunto de componentes, que se detalla a continuación y que se corresponde con lo previsto en la propuesta.

Alejo Ordoñez (Padrón 108397). Núcleo de aprendizaje automático: el modelo, las capas, los optimizadores y las funciones de pérdida. Las capas convolucionales y de *max-pooling*, incluida la reformulación de la convolución como multiplicación de matrices que produjo la mejora final de rendimiento del motor. El manejo y la distribución de los conjuntos de datos. La monografía sobre *Strategy-Switch*, que constituye el tercer estudio experimental del informe.

Lorenzo Minervino (Padrón 107863). El módulo del Worker y la estrategia *All-Reduce* en anillo. La interfaz de funciones externas en Python y la interfaz interactiva de terminal. La suite de benchmarks y el análisis experimental de resultados. La monografía comparativa entre *Parameter Server* y *All-Reduce*, que constituye el primer estudio experimental del informe.

Marcos Bianchi Fernández (Padrón 108921). Los algoritmos de entrenamiento distribuido y su coordinación. El módulo de comunicaciones y el protocolo entre nodos, incluidas las técnicas de reducción de tráfico. El módulo del Servidor de parámetros. La monografía sobre el impacto de las épocas offline, que constituye el segundo estudio experimental del informe.

Cuadro 12: Distribución del trabajo individual por tipo de tarea.

	Alejo Ordoñez	Lorenzo Minervino	Marcos Bianchi Fernández
Cantidad de horas insumidas	<i>[a completar]</i>	<i>[a completar]</i>	<i>[a completar]</i>
Preparación de la propuesta	X	X	X
Relevamiento del estado del arte	X	X	X
Diseño de la arquitectura del sistema	X	X	X
Capa de comunicaciones y protocolo			X
Motor de redes neuronales	X		
Capas convolucionales y max-pooling	X		
Manejo y distribución de datasets	X		
Parameter Server			X

	Alejo Ordoñez	Lorenzo Minervino	Marcos Bianchi Fernández
Worker y All-Reduce en anillo		X	
Strategy-Switch		X	X
Orchestrator y coordinación		X	X
Optimizaciones de comunicación			X
Optimizaciones de sincronización		X	X
Optimización del motor de aprendizaje	X		
Interfaz de funciones externas en Python		X	
Interfaz interactiva de terminal		X	
Entornos de simulación con Docker		X	X
Definición de casos de prueba	X	X	X
Revisión de código	X	X	X
Suite de benchmarks		X	
Análisis experimental de resultados		X	
Monografía: All-Reduce frente a Parameter Server		X	
Monografía: impacto de las épocas offline			X
Monografía: Strategy-Switch	X		
Documentación técnica del sistema	X	X	X
Elaboración del Informe Final	X	X	X

18 Referencias

- Aji, A. F., & Heafield, K. (2017). Sparse Communication for Distributed Gradient Descent. *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, 440-445. <https://doi.org/10.18653/v1/D17-1045>
- Ben-Nun, T., & Hoeffler, T. (2019). Demystifying Parallel and Distributed Deep Learning: An In-Depth Concurrency Analysis. *ACM Computing Surveys*, 52(4). <https://doi.org/10.1145/3320060>
- Dehghani, M., & Yazdanparast, Z. (2023). From Distributed Machine to Distributed Deep Learning: A Comprehensive Survey. *arXiv preprint arXiv:2307.05232*. Recuperado de <https://arxiv.org/abs/2307.05232>
- Developers, T. (2021). *TensorFlow*. Zenodo. <https://doi.org/10.5281/zenodo.4758419>
- Glorot, X., & Bengio, Y. (2010). Understanding the Difficulty of Training Deep Feedforward Neural Networks. *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics*, 249-256.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. *Proceedings of the IEEE International Conference on Computer Vision*, 1026-1034.
- Jiang, Y., Zhu, Y., Lan, C., Yi, B., Cui, Y., & Guo, C. (2020). A Unified Architecture for Accelerating Distributed DNN Training in Heterogeneous GPU/CPU Clusters. *Proceedings of OSDI '20*.
- Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. *arXiv preprint arXiv:1412.6980*. Recuperado de <https://arxiv.org/abs/1412.6980>
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.
- Li, M., Andersen, D. G., Park, J. W., Smola, A. J., Ahmed, A., Josifovski, V., ... Su, B.-Y. (2014). Scaling distributed machine learning with the parameter server. *Proceedings of the 11th USENIX Conference on Operating Systems Design and Implementation*, 583-598. USA: USENIX Association.
- Li, Z., Davis, J., & Jarvis, S. (2017). An Efficient Task-based All-Reduce for Machine Learning Applications. *Proceedings of the Machine Learning on HPC Environments*. New York, NY, USA: Association for Computing Machinery. <https://doi.org/10.1145/3146347.3146350>
- Lin, Y., Han, S., Mao, H., Wang, Y., & Dally, W. J. (2018). Deep Gradient Compression: Reducing the Communication Bandwidth for Distributed Training. *International Conference on Learning Representations*. Recuperado de <https://arxiv.org/abs/1712.01887>
- McMahan, B., Moore, E., Ramage, D., Hampson, S., & Arcas, B. A. y. (2017). Communication-Efficient Learning of Deep Networks from Decentralized Data. *Artificial Intelligence and Statistics*, 1273-1282. PMLR.
- Niu, F., Recht, B., Ré, C., & Wright, S. J. (2011). Hogwild!: A Lock-Free Approach to Parallelizing Stochastic Gradient Descent. *Advances in Neural Information Processing Systems 24*. Recuperado de <https://arxiv.org/abs/1106.5730>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... Chintala, S. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Recuperado de <https://arxiv.org/abs/1912.01703>
- Patarasuk, P., & Yuan, X. (2009). Bandwidth Optimal All-reduce Algorithms for Clusters of Workstations. *Journal of Parallel and Distributed Computing*, 69(2), 117-124. <https://doi.org/10.1016/j.jpdc.2008.09.002>
- Prechelt, L. (1998). Early Stopping — But When? En *Neural Networks: Tricks of the Trade* (pp.

- 55-69). Springer. https://doi.org/10.1007/3-540-49430-8_3
- Provatas, N., Chalas, I., Konstantinou, I., & Koziris, N. (2025). Strategy-Switch: From All-Reduce to Parameter Server for Faster Efficient Training. *IEEE Access, PP*, 1-1. <https://doi.org/10.1109/ACCESS.2025.3528248>
- Sergeev, A., & Del Balso, M. (2018). Horovod: Fast and Easy Distributed Deep Learning in TensorFlow. *arXiv preprint arXiv:1802.05799*. Recuperado de <https://arxiv.org/abs/1802.05799>
- Sutskever, I., Martens, J., Dahl, G., & Hinton, G. (2013). On the Importance of Initialization and Momentum in Deep Learning. *Proceedings of the 30th International Conference on Machine Learning*, 1139-1147.
- Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. *arXiv preprint arXiv:1708.07747*. Recuperado de <https://arxiv.org/abs/1708.07747>

19 Anexos

19.1 Anexo A: Repositorio del proyecto

El código fuente del trabajo, junto con su documentación técnica y el historial completo de su desarrollo, se encuentra alojado en el siguiente repositorio público, que se mantendrá disponible al menos hasta un año después de la defensa:

<https://github.com/lminervino18/oxidized-neural-orchestra>

El repositorio está organizado como un *workspace* de Cargo, la herramienta de construcción de Rust, y se divide en los siguientes módulos:

- **machine_learning**: el núcleo de redes neuronales, que comprende las capas, los optimizadores, las funciones de pérdida y la representación de tensores.
- **comms**: el middleware de comunicación entre nodos.
- **parameter_server**: la implementación de la estrategia *Parameter Server*.
- **worker**: el runtime del nodo trabajador, que contiene además la implementación en anillo de *All-Reduce*.
- **orchestrator**: el plano de control, que asigna los roles de servidor y de trabajador y dirige la ejecución de un entrenamiento.
- **node**: el proceso de cómputo, agnóstico del rol que le sea asignado.
- **orchestra-py**: la biblioteca de Python, construida sobre la interfaz de funciones externas.
- **orchestui**: la interfaz interactiva de terminal.
- **benchmarks**: la suite de experimentación y los generadores de gráficos.
- **docker**: los scripts que levantan los entornos de simulación multinodo.
- **docs**: la propuesta, el presente informe y la documentación de diseño del sistema.

19.2 Anexo B: Documentación técnica

Dentro del repositorio se encuentran disponibles los siguientes documentos, referenciados a lo largo de este informe:

- **docs/config-schema.md**: la referencia canónica del esquema de configuración de un entrenamiento. Es la única fuente de verdad del formato aceptado por las tres interfaces del sistema.
- **docs/architecture-draft.md**: el borrador de arquitectura que sostuvo las discusiones de diseño de las etapas iniciales.
- **docs/report/roadmap.md**: la reconstrucción detallada del recorrido conceptual del proyecto a partir del historial completo del repositorio, con las decisiones de arquitectura, los debates que las motivaron y su fundamentación bibliográfica. Es el documento de apoyo del que se derivan las secciones de *Metodología aplicada*, *Cronograma* y *Riesgos materializados y lecciones aprendidas*.
- **benchmarks/README.md**: la documentación de la suite de experimentación, con las configuraciones de cada campaña y las instrucciones para reproducirlas.
- Los **README.md** de cada crate, con la documentación de sus abstracciones principales.

19.3 Anexo C: Reproducibilidad de los experimentos

Los estudios experimentales presentados en este informe son reproducibles a partir del material versionado en el repositorio. Para cada campaña se conservan las configuraciones que la generaron, los resultados crudos y los resultados procesados, junto con los scripts que producen las figuras y las tablas a partir de ellos.

El material del primer estudio se encuentra en `doc/monography_all_reduce_vs_ps/`, e incluye el arnés de experimentación autocontenido, los archivos de configuración de los cuatro experimentos, los resultados en formato crudo y procesado, y los generadores de figuras y tablas. La campaña completa de benchmarks del sistema se ejecuta mediante `benchmarks/run_issue_benchmarks.py`; su última corrida completa abarcó 38 configuraciones sin fallos, con un tiempo total de cómputo de 5 horas y 39 minutos, y con tres repeticiones en las suites de throughput y de escalabilidad para reportar media y desvío.

Las semillas de inicialización de los parámetros están fijadas en todas las configuraciones. Debe tenerse presente, no obstante, que las mediciones de tiempo dependen del hardware y de la carga de la máquina en la que se ejecuten, de modo que la reproducibilidad exacta alcanza a las métricas de convergencia y de exactitud, y no a las de rendimiento.

19.4 Anexo D: Monografías

Los dos primeros estudios experimentales de este informe fueron desarrollados originalmente como monografías individuales en el marco de la materia Aprendizaje Profundo, y se presentan aquí en forma condensada y reorientada al sistema. Los documentos completos, con su desarrollo extendido, se encuentran disponibles en el repositorio.